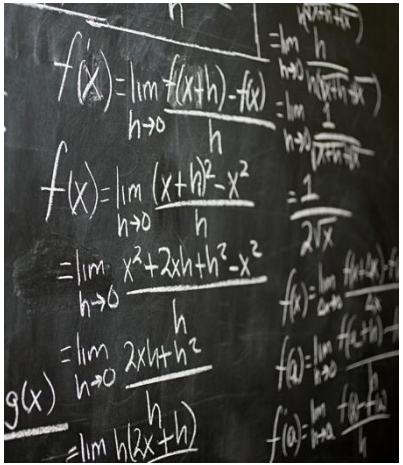
Devito DSL Paradigm: Experiences and Lessons in HPC Development and Workflow Integration

Devito Workshop: Energy HPC & AI Conference 2026

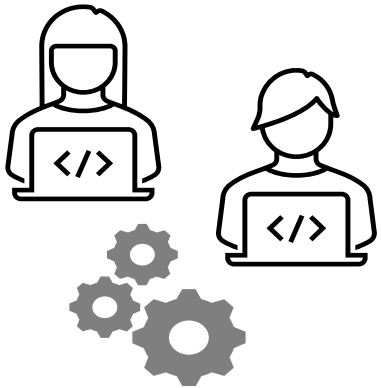
Thomas Cullison, Geophysics PhD Student

Once upon a time…

PDE Model



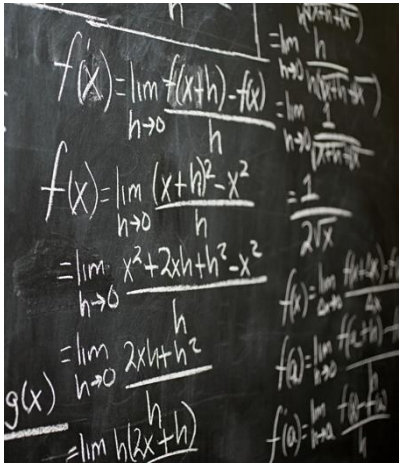
HPC Development



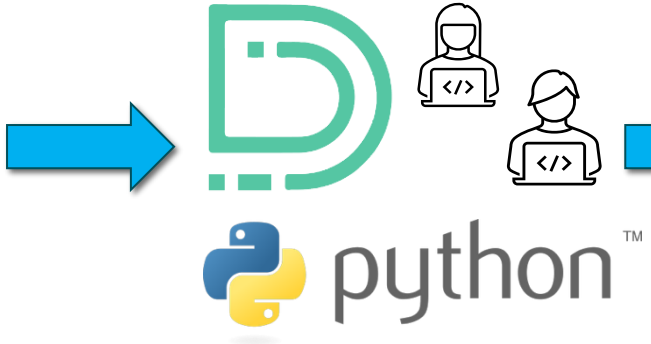
HPC Application

Some years later…

PDE Model



HPC Development

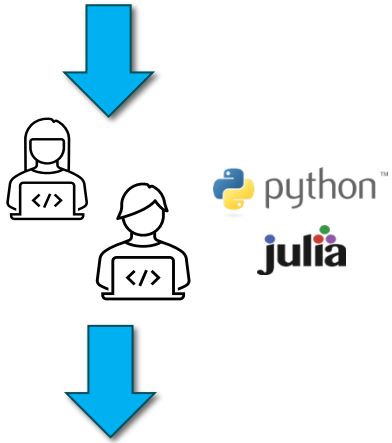


HPC Application

Adapting to a Different Paradigm

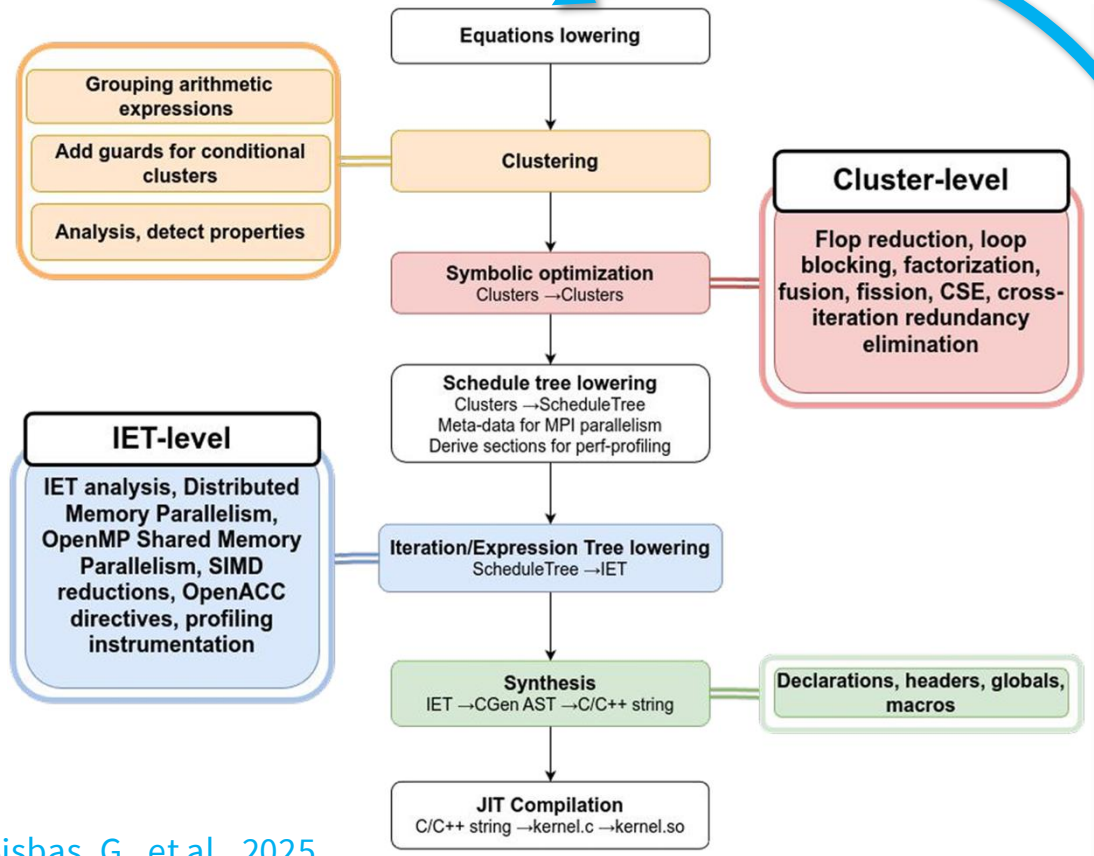
Devito Overview

$$m\frac{\partial^2 u}{\partial t^2} + \eta\frac{\partial u}{\partial t} - \Delta u = 0$$



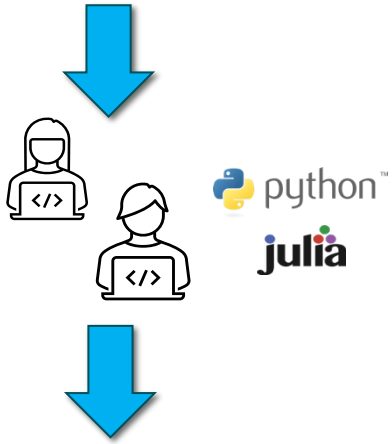
eq = m*u.dt2 + eta*u.dt - u.laplace

Devito Overview



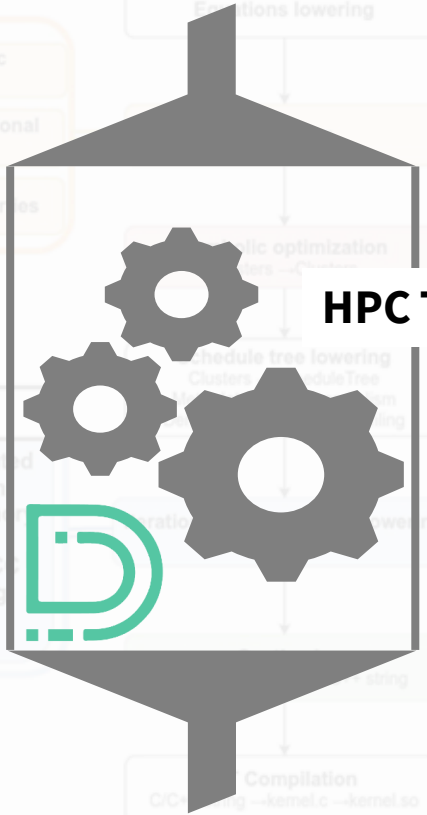
Bisbas, G., et al., 2025

$$m\frac{\partial^2 u}{\partial t^2} + \eta\frac{\partial u}{\partial t} - \Delta u = 0$$

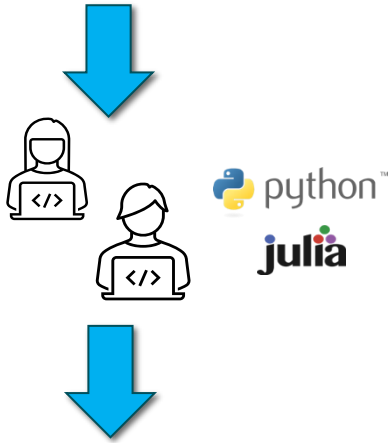



```
eq = m*u.dt2 + eta*u.dt - u.laplace
```

Devito Overview

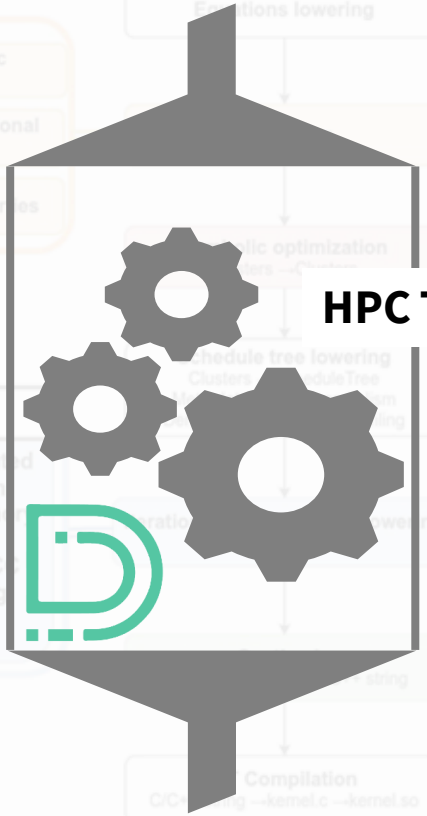


$$m\frac{\partial^2 u}{\partial t^2} + \eta\frac{\partial u}{\partial t} - \Delta u = 0$$

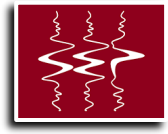



```
eq = m*u.dt2 + eta*u.dt - u.laplace
```

Devito Overview



HPC Application



Devito Overview

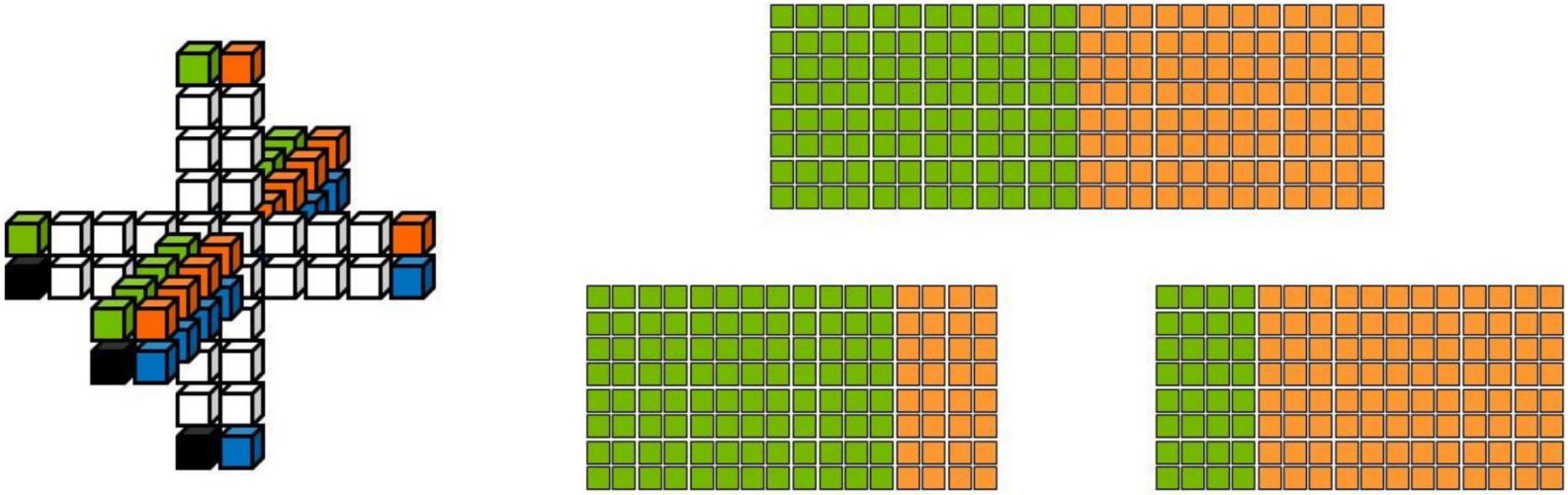
Partially

HPC Application

One Must Still Build the App

Automated HPC Developer Tasks

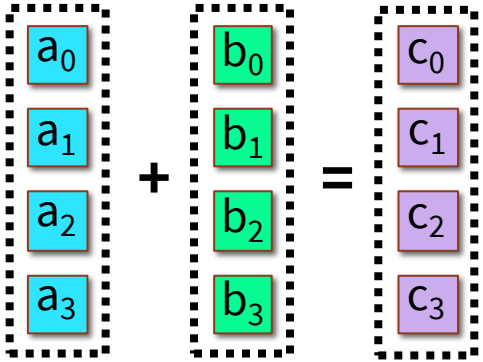
- FD stencil-loops and indexing in space/time
  - Various space/time orders
  - Boundary or decomposition loops (SubDomains)



Micikevicius, P., NVIDIA, 2009

Automated HPC Developer Tasks

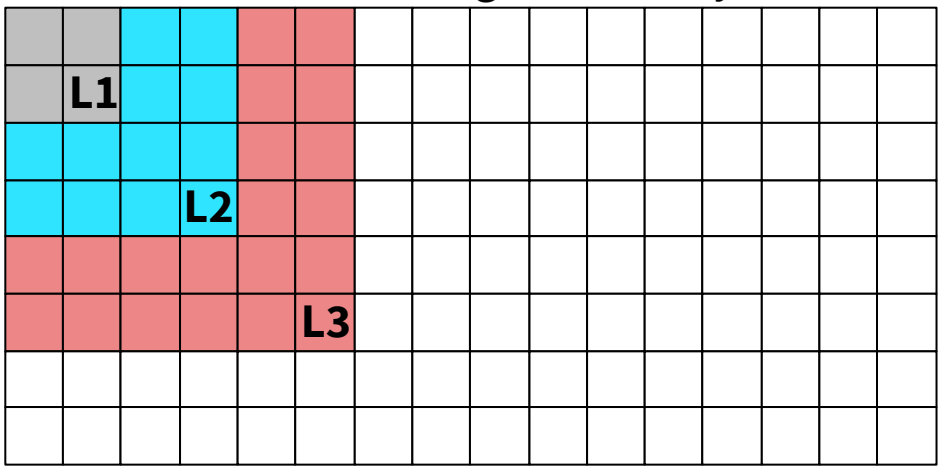
- FD stencil-loops and indexing in space/time
  - Various space/time orders
  - Boundary or decomposition loops (SubDomains)
- SIMD-friendly code and vectorization



Automated HPC Developer Tasks

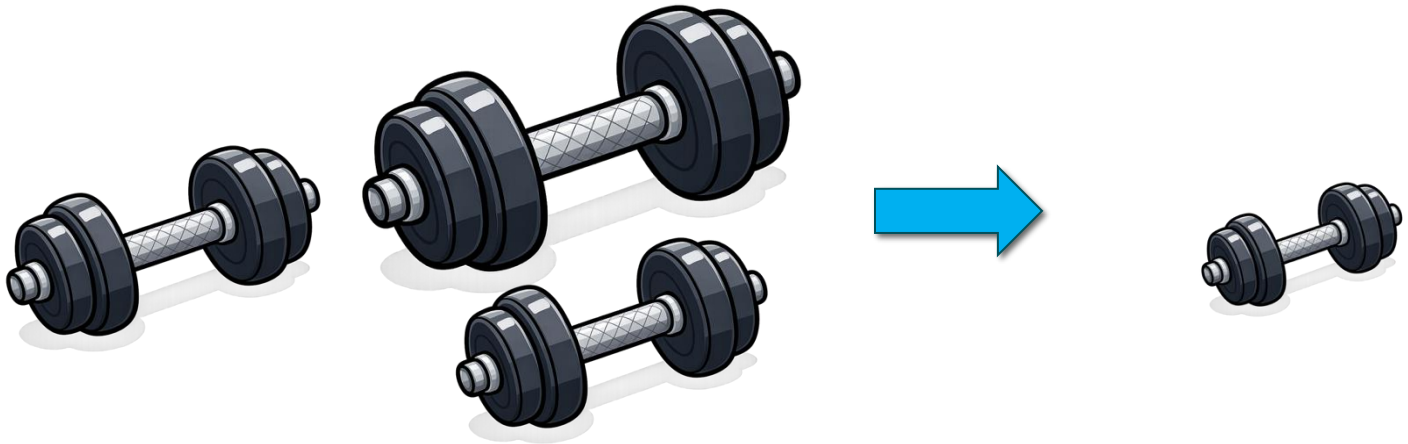
- FD stencil-loops and indexing in space/time
  - Various space/time orders
  - Boundary or decomposition loops (SubDomains)
- SIMD-friendly code and vectorization
- Cache blocking/tiling including shape and size

Cache Blocking for Memory



Automated HPC Developer Tasks

- FD stencil-loops and indexing in space/time
  - Various space/time orders
  - Boundary or decomposition loops (SubDomains)
- SIMD-friendly code and vectorization
- Cache blocking/tiling including shape and size
- CSE, flop reduction, loop-invariant hoisting

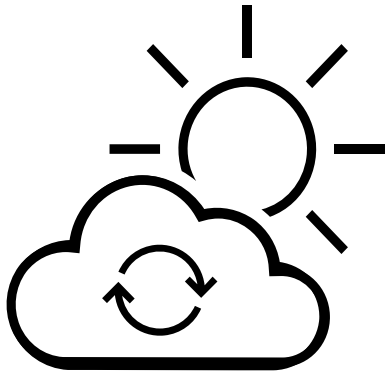


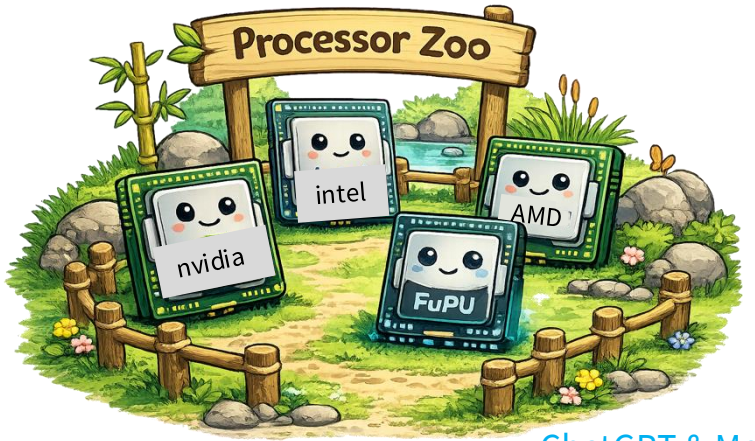
Automated HPC Developer Tasks

- FD stencil-loops and indexing in space/time
  - Various space/time orders
  - Boundary or decomposition loops (SubDomains)
- SIMD-friendly code and vectorization
- Cache blocking/tiling including shape and size
- CSE, flop reduction, loop-invariant hoisting
- OpenMP/OpenACC/CUDA/HIP

Automated HPC Developer Tasks

- FD stencil-loops and indexing in space/time
  - Various space/time orders
  - Boundary or decomposition loops (SubDomains)
- SIMD-friendly code and vectorization
- Cache blocking/tiling including shape and size
- CSE, flop reduction, loop-invariant hoisting
- OpenMP/OpenACC/CUDA/HIP
- Architecture/language specific compilation
  - Same Python code: CPUs and GPUs
  - *Future*-PUs?

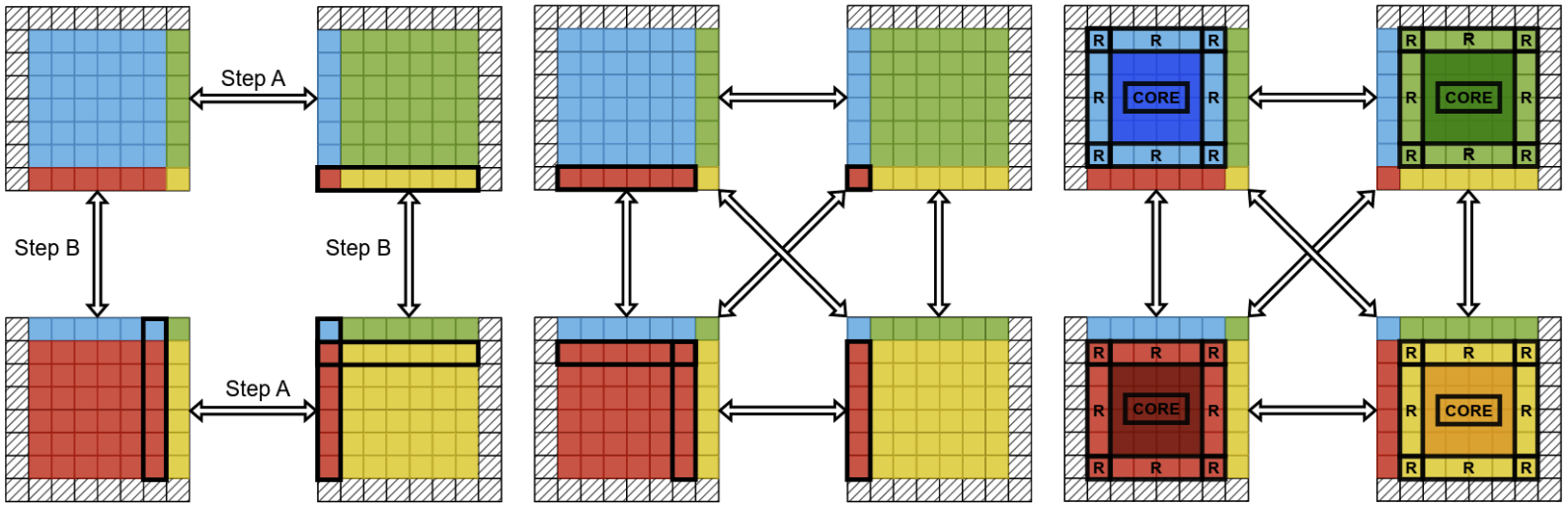




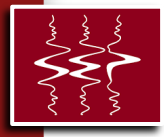
ChatGPT & Me

Including MPI

- MPI domain decomposition and halo-exchange



Bisbas, G., et al., 2025

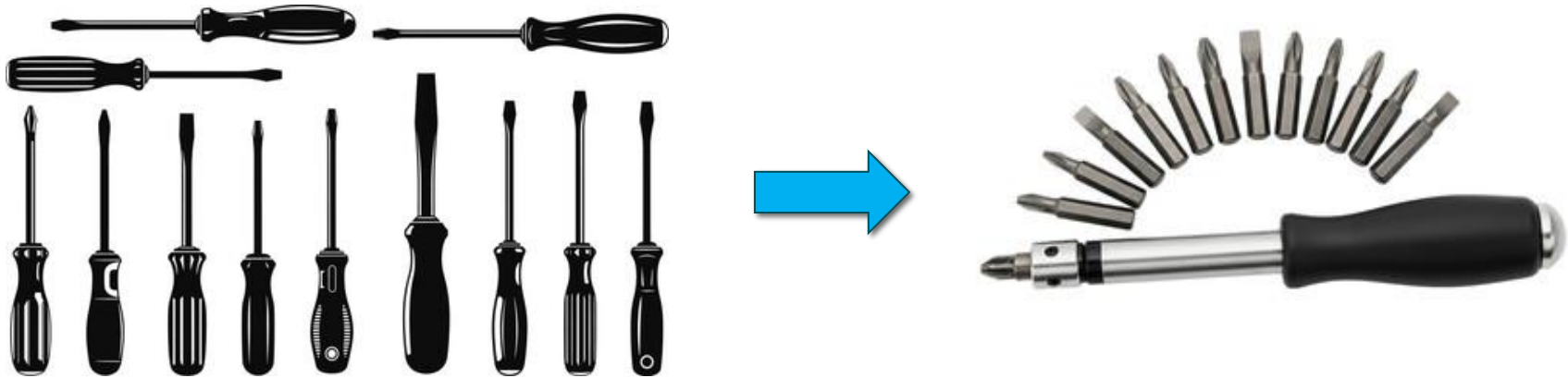


What Is Not Automated

- Validation and quality control
- Integration into HPC workflows
- System-level performance

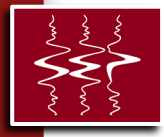
Adapting to Different Development Paradigm

- Validation and quality control
- Integration into HPC workflows
- System-level performance



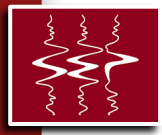
Adapting to Different Development Paradigm

- Validation and quality control
- Integration into HPC workflows
- System-level performance



Adapting to Different Development Paradigm

- Validation and quality control
- Integration into HPC workflows
- System-level performance



Adapting to Different Development Paradigm

- Validation and quality control
- Integration into HPC workflows
- System-level performance

Step 1:

devito/examples/userapi/00_sympy.ipynb

Adapting to Different Development Paradigm

- Validation and quality control
- Integration into HPC workflows
- System-level performance

Adapting to Different Development Paradigm

- Validation and quality control
- Integration into HPC workflows
- System-level performance

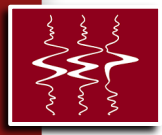
Acoustic and Elastic Wave Equations

Validation and quality control

- Stability
- Result analysis

Validation and quality control

- Stability
  - Positioning (indexing and domain decomposition)
  - Stencils
  - CFL
- Result analysis
  - Are the results consistent with expectations
    - Boundaries
    - Changes in source and receiver (filter function) locations
    - Receiver data agrees with final wavefields (state parameters)
    - Changes in medium parameters



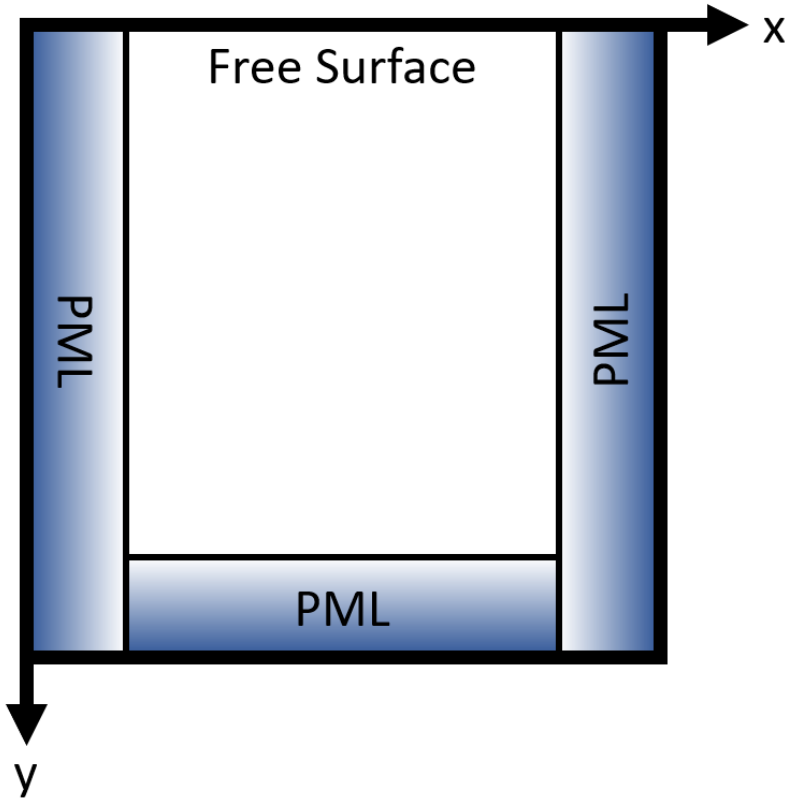
Validation and quality control

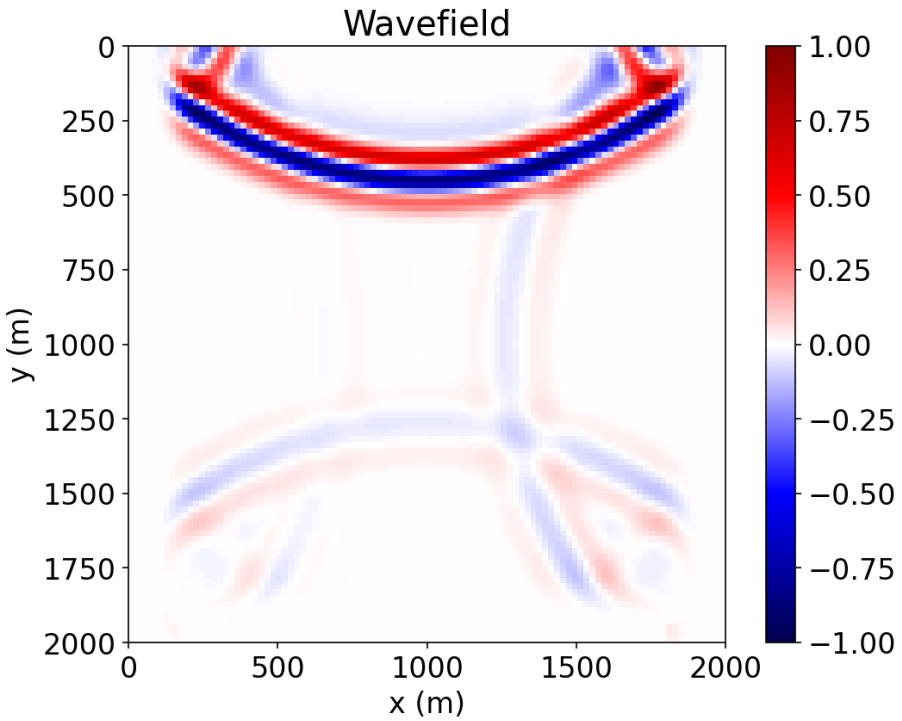
- Stability
  - Positioning (indexing and domain decomposition)
  - Stencils
  - CFL
- Result analysis
  - Are the results consistent with expectations
    - Boundaries
    - Changes in source and receiver (filter function) locations
    - Receiver data agrees with final wavefields (state parameters)
    - Changes in medium parameters

Validation and quality control

- Stability
  - Positioning (indexing and domain decomposition)
  - Stencils
  - CFL
- Result analysis
  - Are the results consistent with expectations
    - Boundaries
    - Changes in source and receiver (filter function) locations
    - Receiver data agrees with final wavefields (state parameters)
    - Changes in medium parameters

Positioning (indexing and domain decomposition)

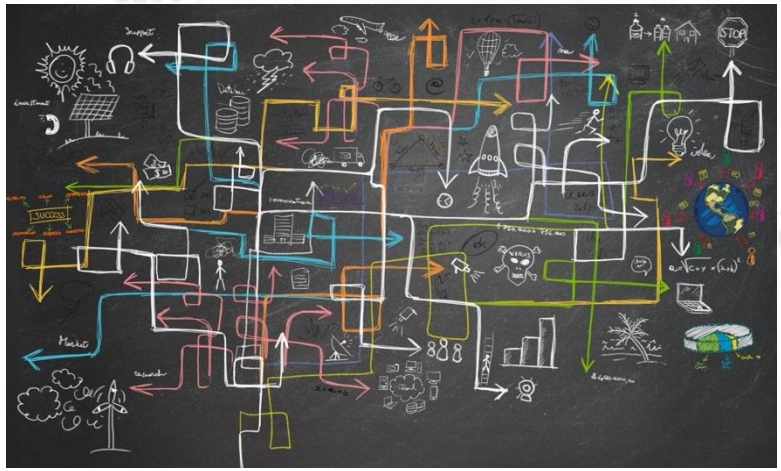


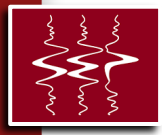


notebooks/SubDomain_QC_04_boundary_domains.ipynb

Positioning (indexing and domain decomposition)

Classic HPC: Loops with "clever" indexing

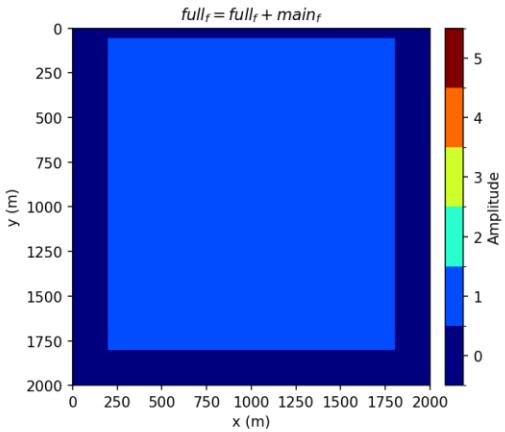


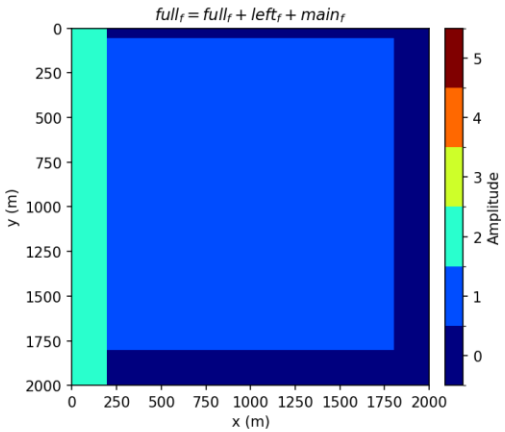


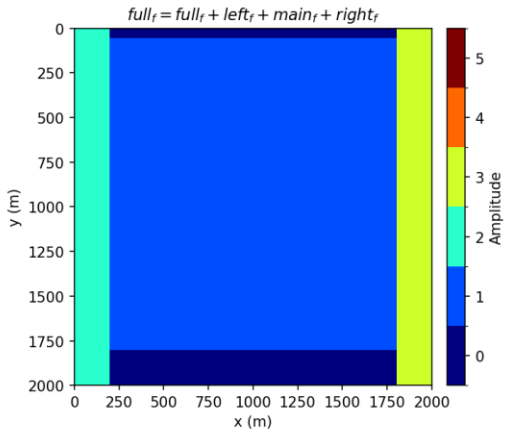
Classic HPC: (nested loops … kernels … functions)

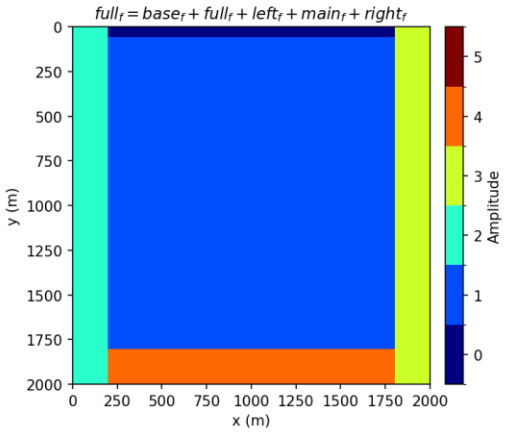
```
// Time loop
for (int t = 0; t < /*NT*/; ++t) {

  // Halo exchange / boundary fill would go here (MPI + pack/unpack)
  // exchange_halos(u0);

  // Cache blocking (tiling) over local interior, expressed in global coords
  for (int gzB = gz0; gzB < gz1; gzB += BZ) {
    int gzE = std::min(gzB + BZ, gz1);

    for (int gyB = gy0; gyB < gy1; gyB += BY) {
      int gyE = std::min(gyB + BY, gy1);

      for (int gxB = gx0; gxB < gx1; gxB += BX) {
        int gxE = std::min(gxB + BX, gx1);

        // Inner loops (candidate for SIMD + unrolling)
        for (int gz = gzB; gz < gzE; ++gz) {
          int z = lz(gz);

          for (int gy = gyB; gy < gyE; ++gy) {
            int y = ly(gy);

            // Domain decomposition shows up as start/end (gxB..gxE)
            // Unrolling shows up as stepping i by U and doing i+0..U-1
            constexpr int U = /* unroll factor, e.g., 4 or 8 */;

            int gx = gxB;
            for (; gx < gxE && (/*alignment or remainder condition*/); ++gx) {
              int x = lx(gx);

              // TODO: stencil update at (x,y,z)
              // u1[idx(x,y,z)] = ...
            }

            // Unrolled core
            for (; gx + (U - 1) < gxE; gx += U) {
              int x0 = lx(gx + 0);
              int x1 = lx(gx + 1);
              int x2 = lx(gx + 2);
              int x3 = lx(gx + 3);

              // TODO: if U>4, add x4..x7, etc.

              // TODO: stencil updates (each is independent)
              // u1[idx(x0,y,z)] = ...
              // u1[idx(x1,y,z)] = ...
              // u1[idx(x2,y,z)] = ...
              // u1[idx(x3,y,z)] = ...
            }

            // Epilogue remainder
            for (; gx < gxE; ++gx) {
              int x = lx(gx);
```

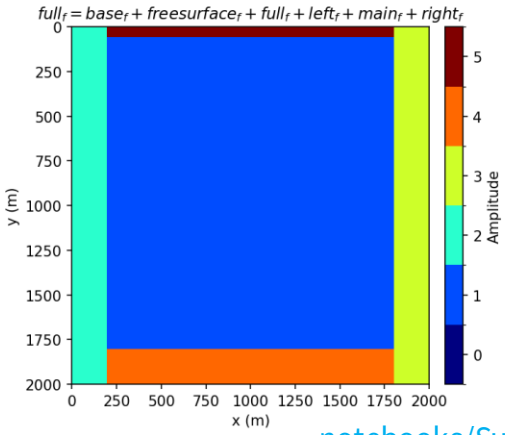
Correct SubDomains





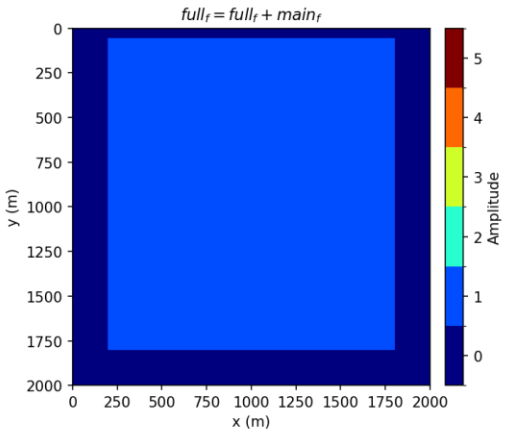


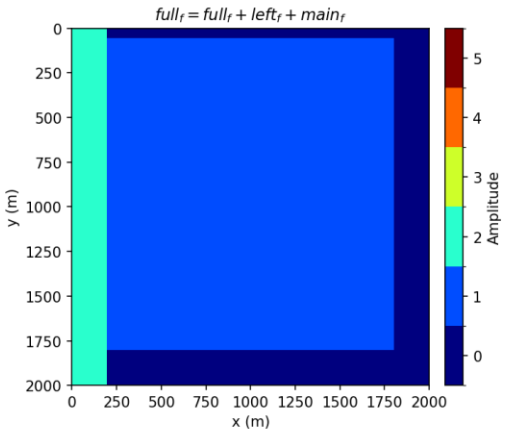


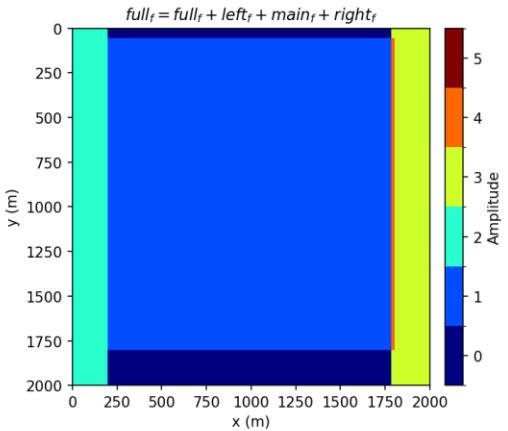


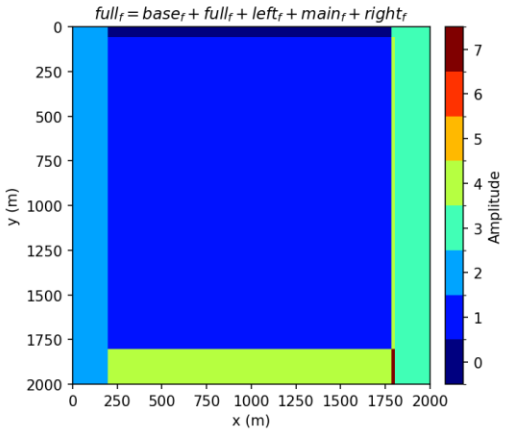
notebooks/SubDomain_QC_04_boundary_domains.ipynb

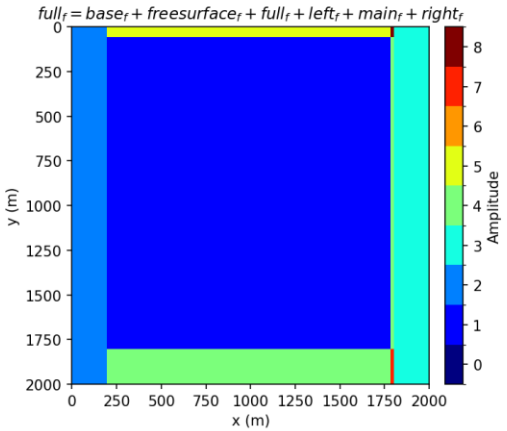
Incorrect SubDomains



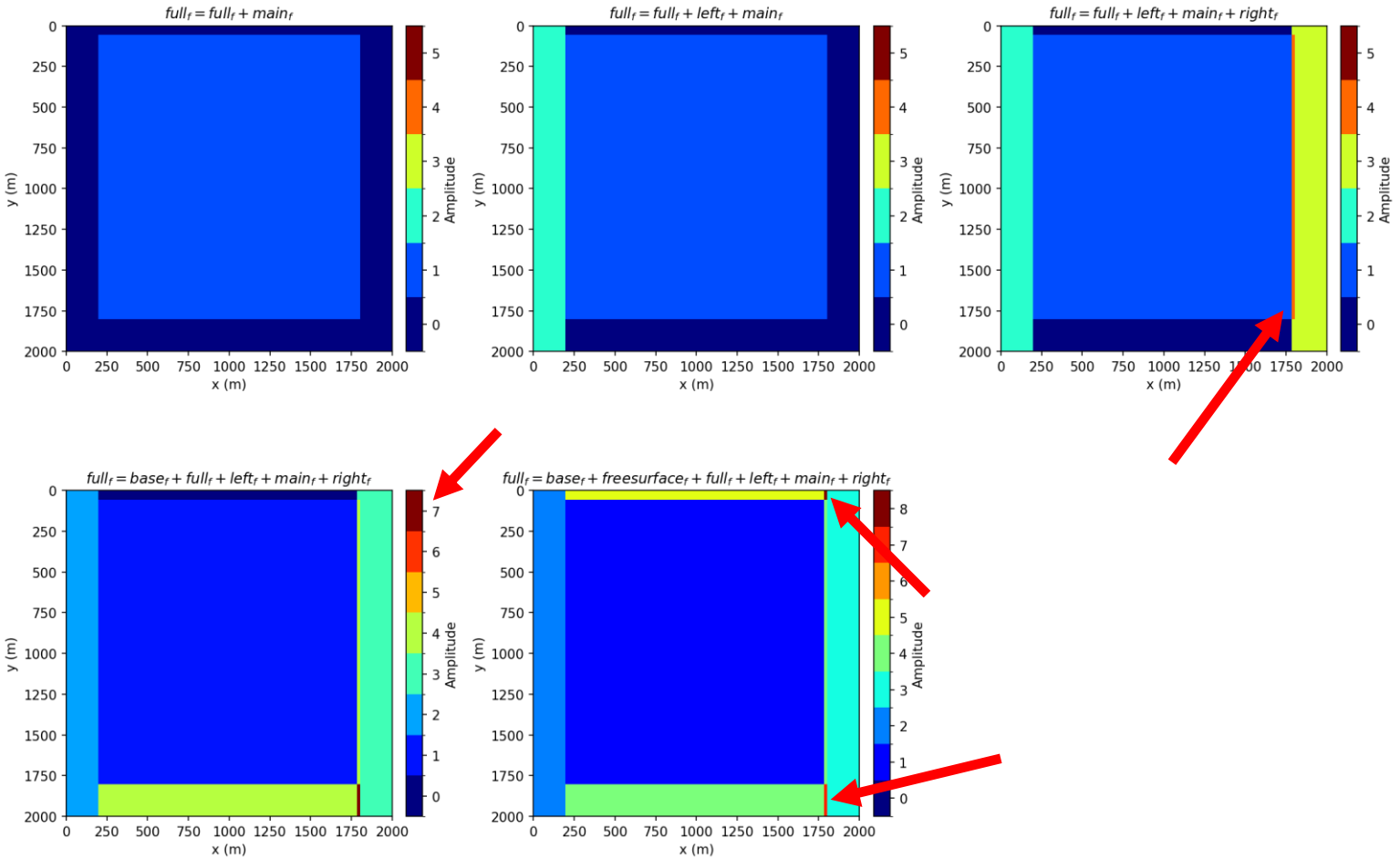




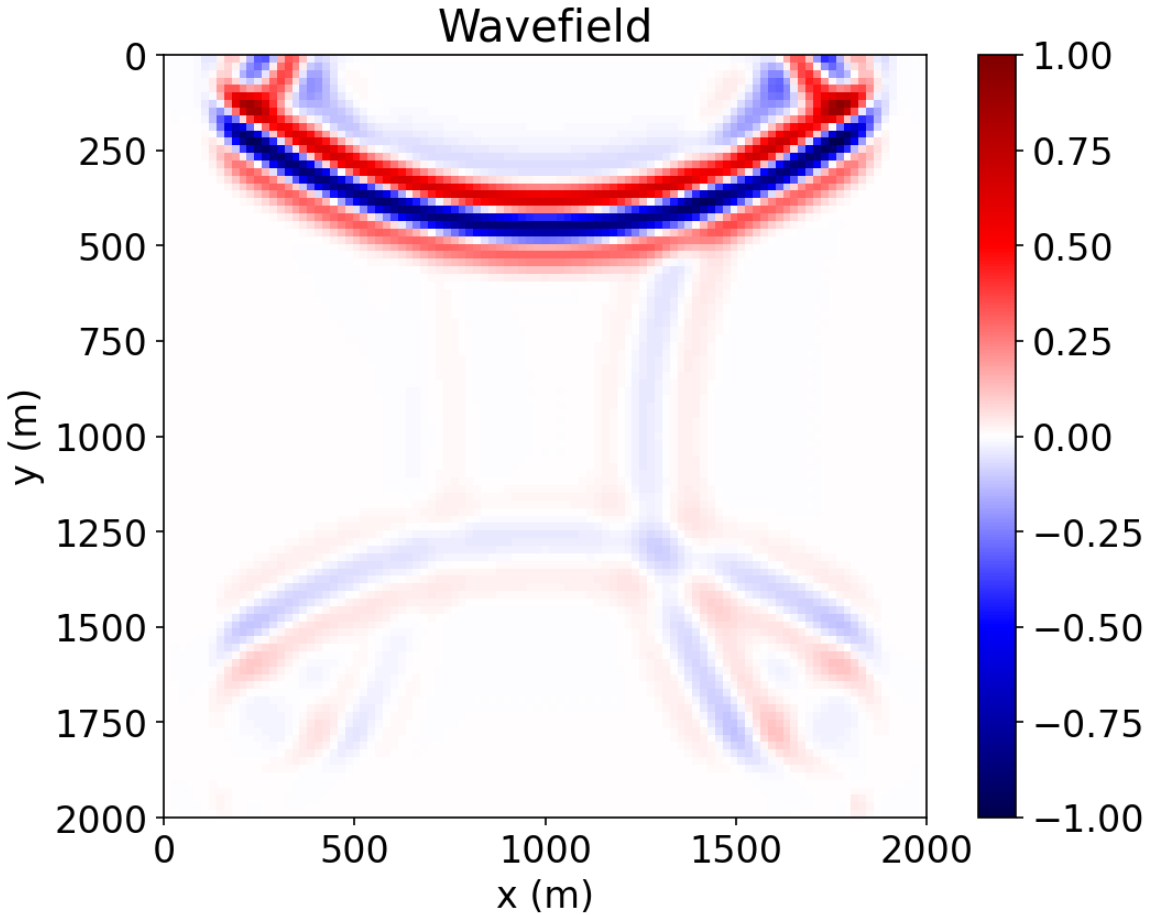




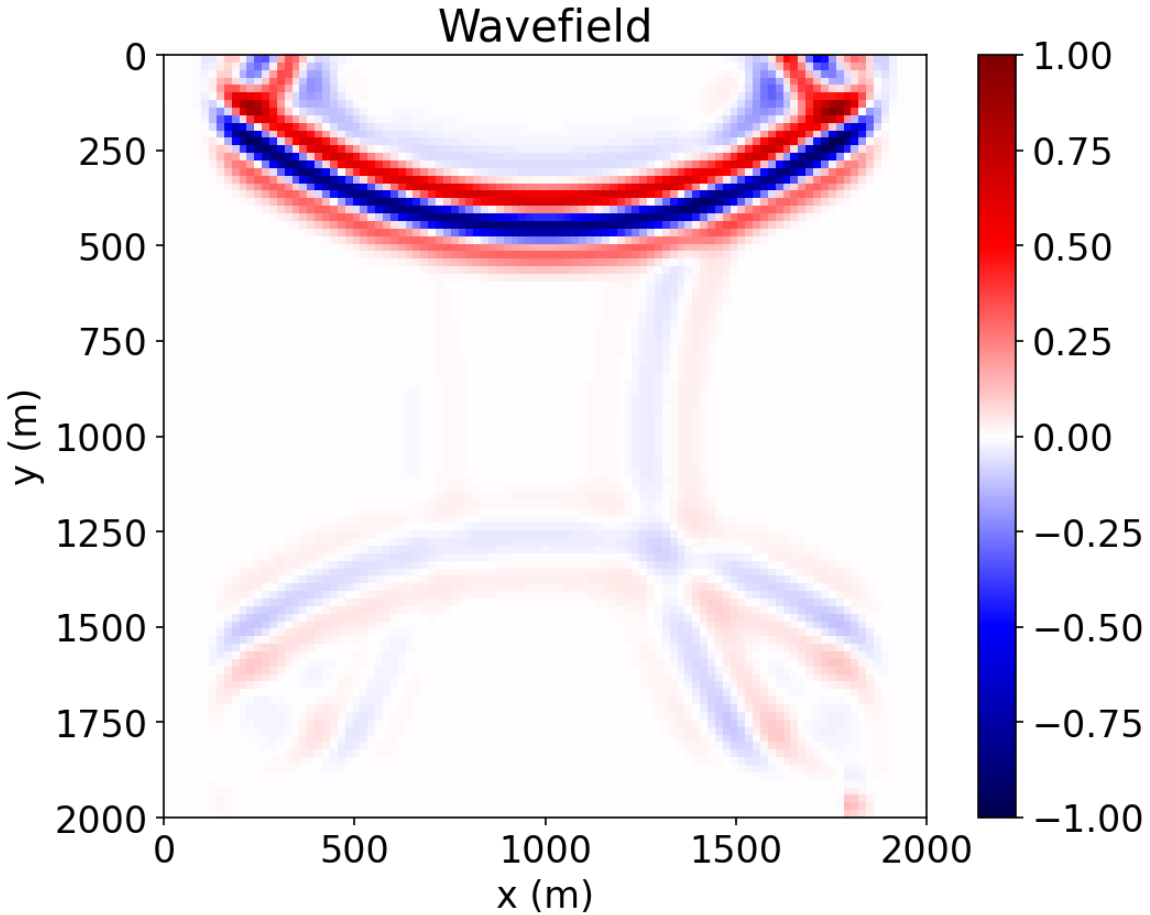
Incorrect SubDomains



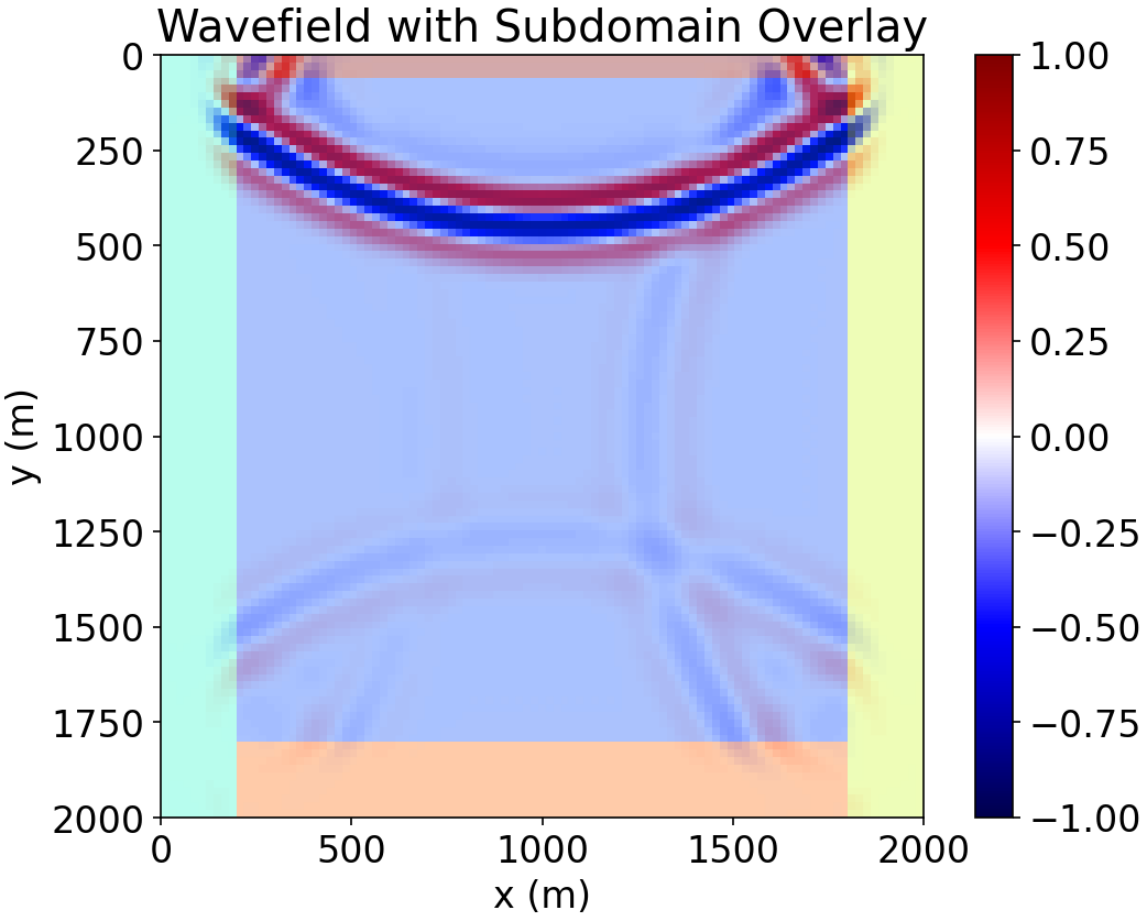
Wavefield: Correct SubDomains



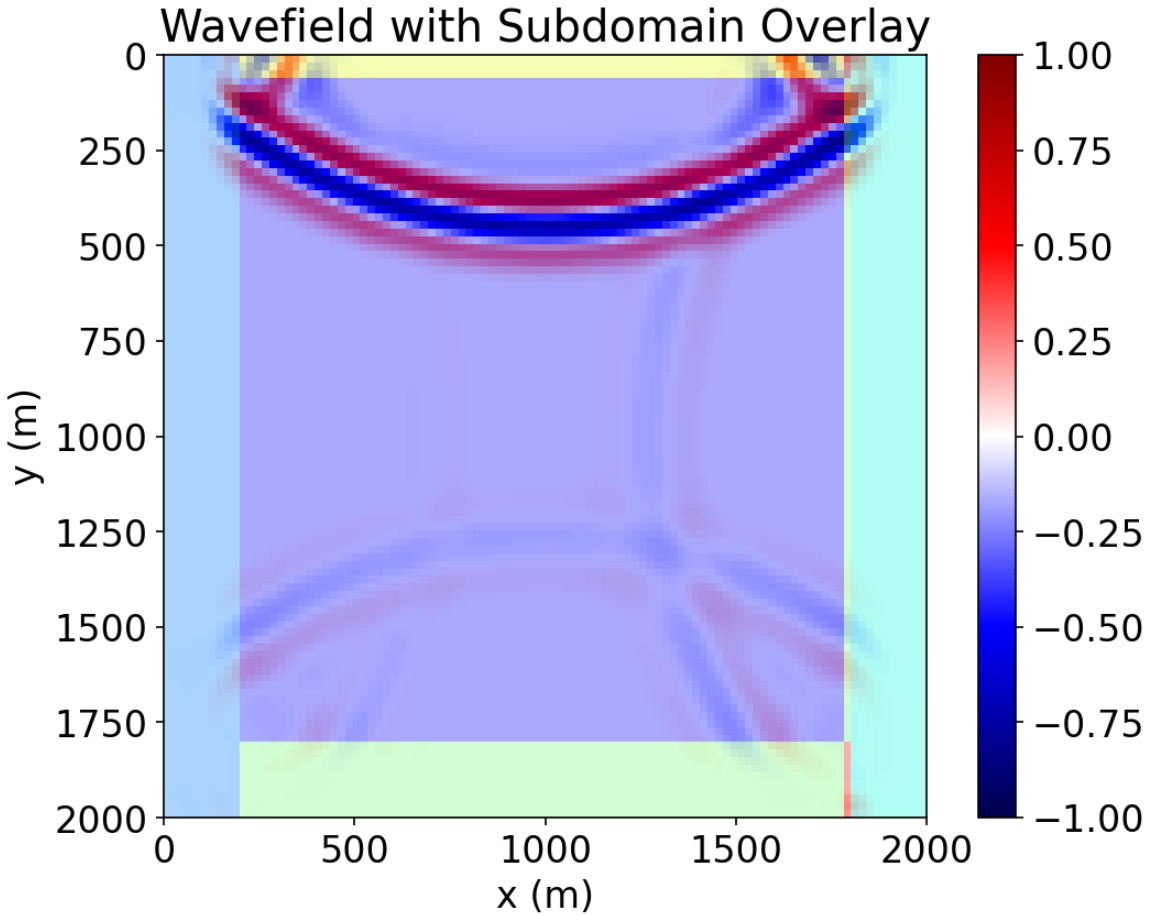
Wavefield: Incorrect SubDomains



Overlay: Correct SubDomains

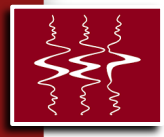


Overlay: Incorrect SubDomains



Validation and quality control

- Stability
  - Positioning (indexing and domain decomposition)
  - Stencils
  - CFL
- Result analysis
  - Are the results consistent with expectations
    - Boundaries
    - Changes in source and receiver (filter function) locations
    - Receiver data agrees with final wavefields (state parameters)
    - Changes in medium parameters



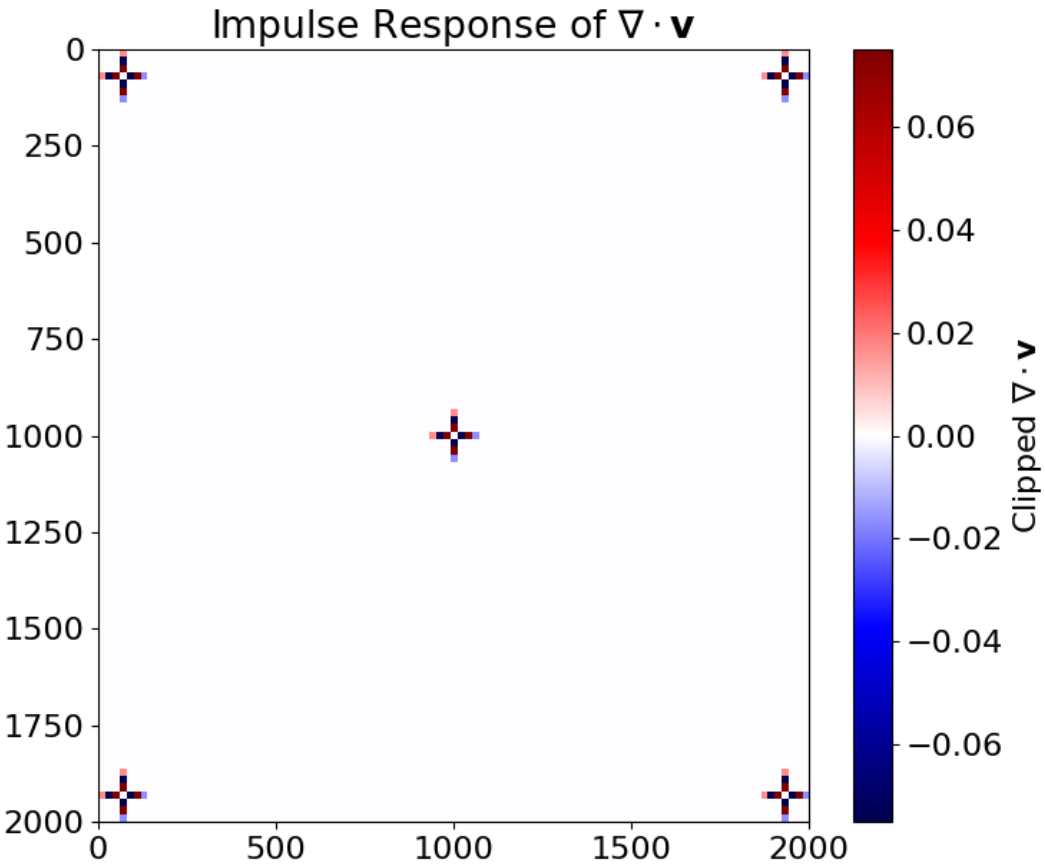
Examining Stencils

```
1  from devito import Eq, grad, div
2
3  eq_v = Eq(v.forward, v - dt*grad(p)/density, subdomain=main)
4
5  eq_p = Eq(p.forward, p - dt*velocity**2*density*div(v.forward), subdomain=main)
```

Examining Stencils

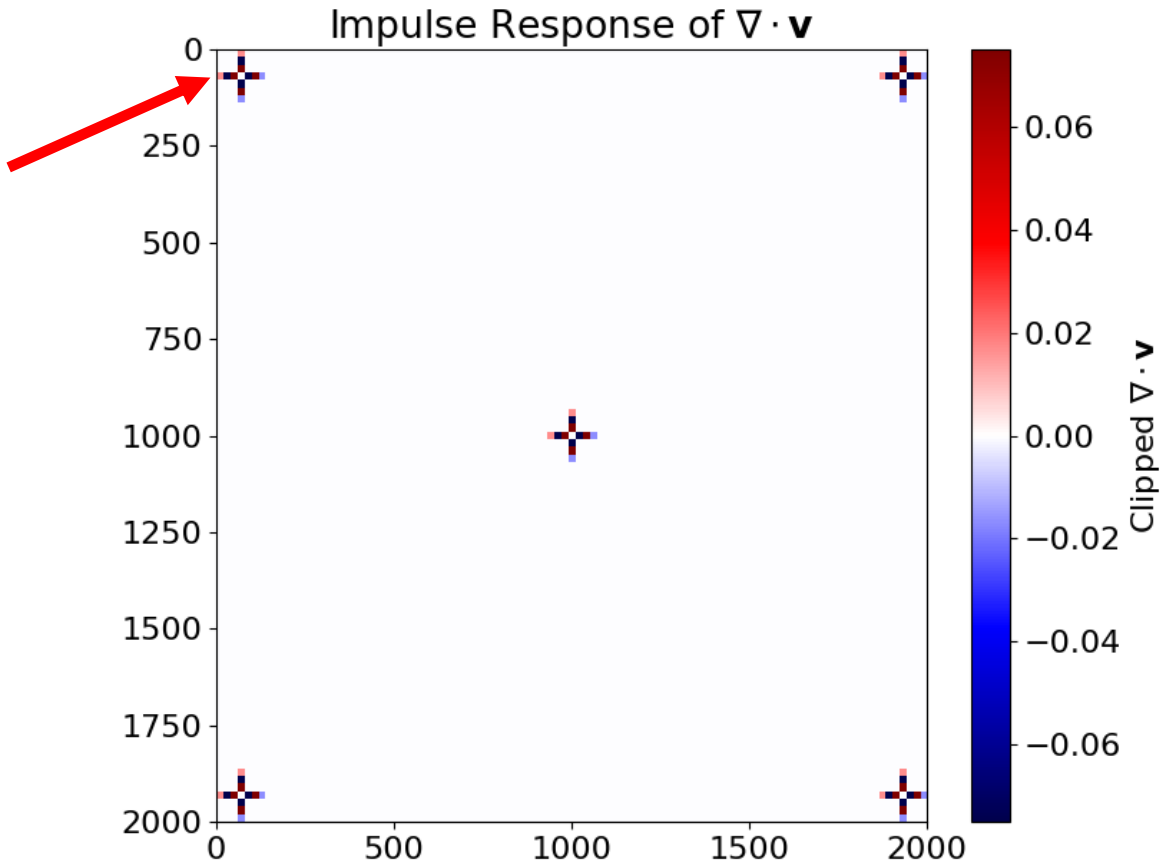
```
1  from devito import Eq, grad, div
2
3  eq_v = Eq(v.forward, v - dt*grad(p)/density, subdomain=main)
4
5  eq_p = Eq(p.forward, p - dt*velocity**2*density*div(v.forward), subdomain=main)
```

Divergence Stencil: Impulse Responses

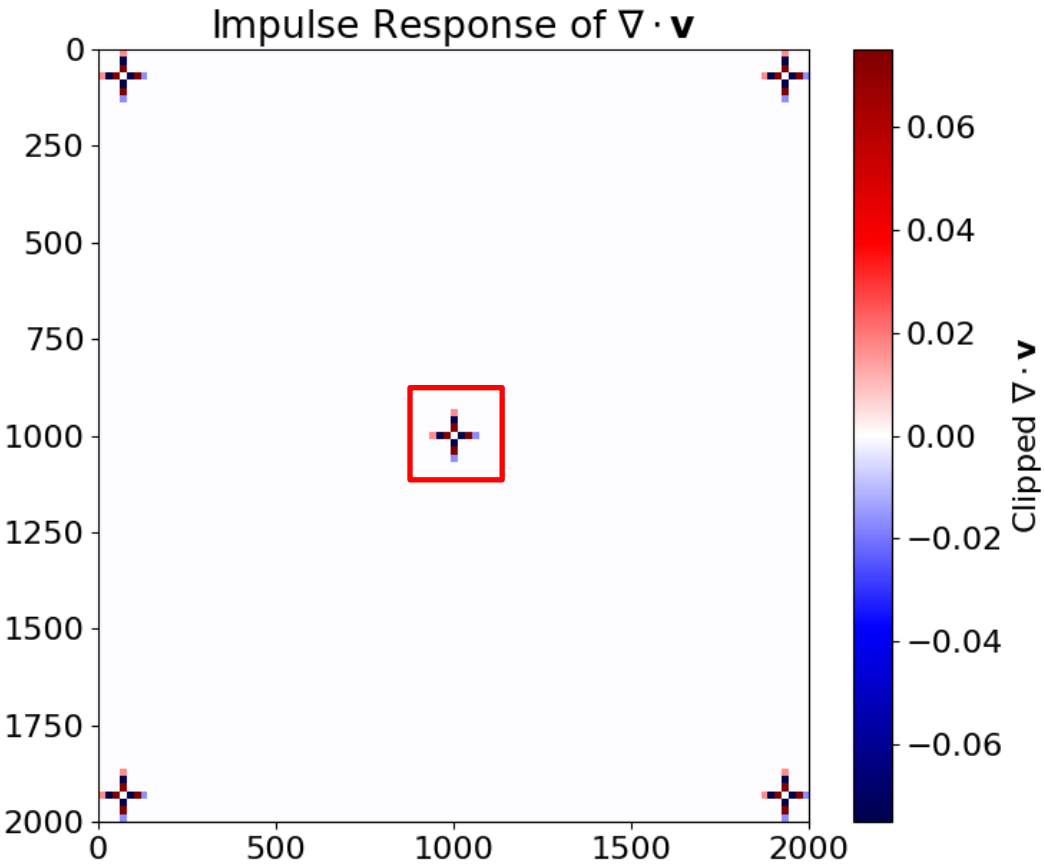


notebooks/SubDomain_QC_Stencils_04_boundary_domains.ipynb

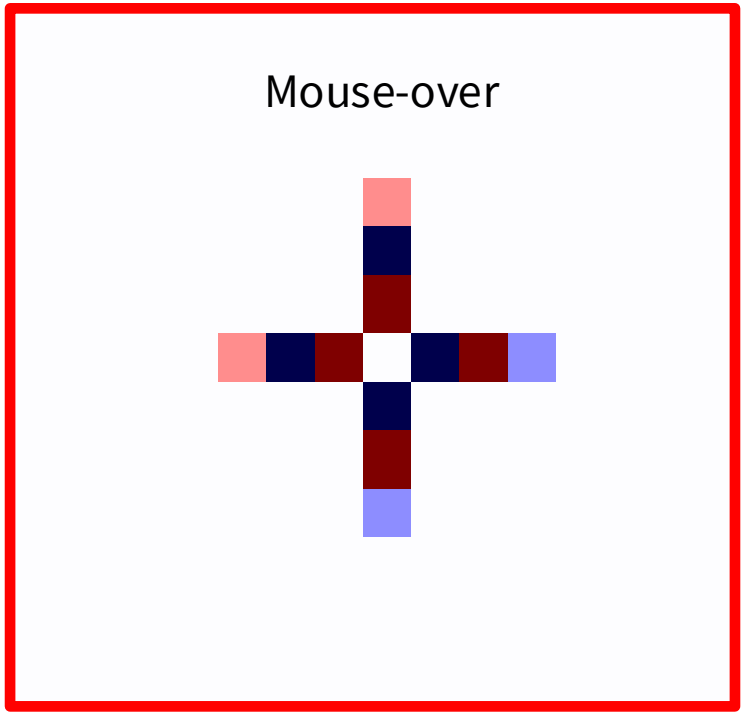
Divergence Stencil: Impulse Responses



Divergence Stencil: Impulse Responses

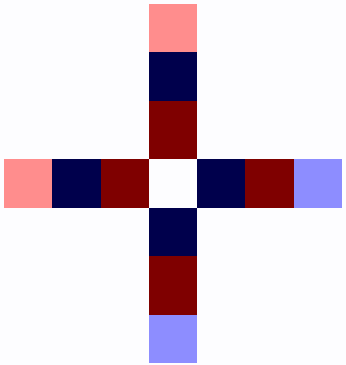


Divergence Stencil: Impulse Responses



Divergence Stencil: Impulse Responses

What Spatial Order?



Divergence Stencil: Impulse Responses

Grid Spacing (h_x and h_y)

	h_x	h_y
0	20.0	20.0

vx Coefficients

	Function	Coefficient	Coefficient/h_x
0	v_x(t + dt, x + 3*h_x/2, y)	0.75/h_x	0.037500
1	v_x(t + dt, x + 7*h_x/2, y)	0.0166666667/h_x	0.000833
2	v_x(t + dt, x - 3*h_x/2, y)	0.15/h_x	0.007500
3	v_x(t + dt, x - h_x/2, y)	-0.75/h_x	-0.037500
4	v_x(t + dt, x - 5*h_x/2, y)	-0.0166666667/h_x	-0.000833
5	v_x(t + dt, x + 5*h_x/2, y)	-0.15/h_x	-0.007500

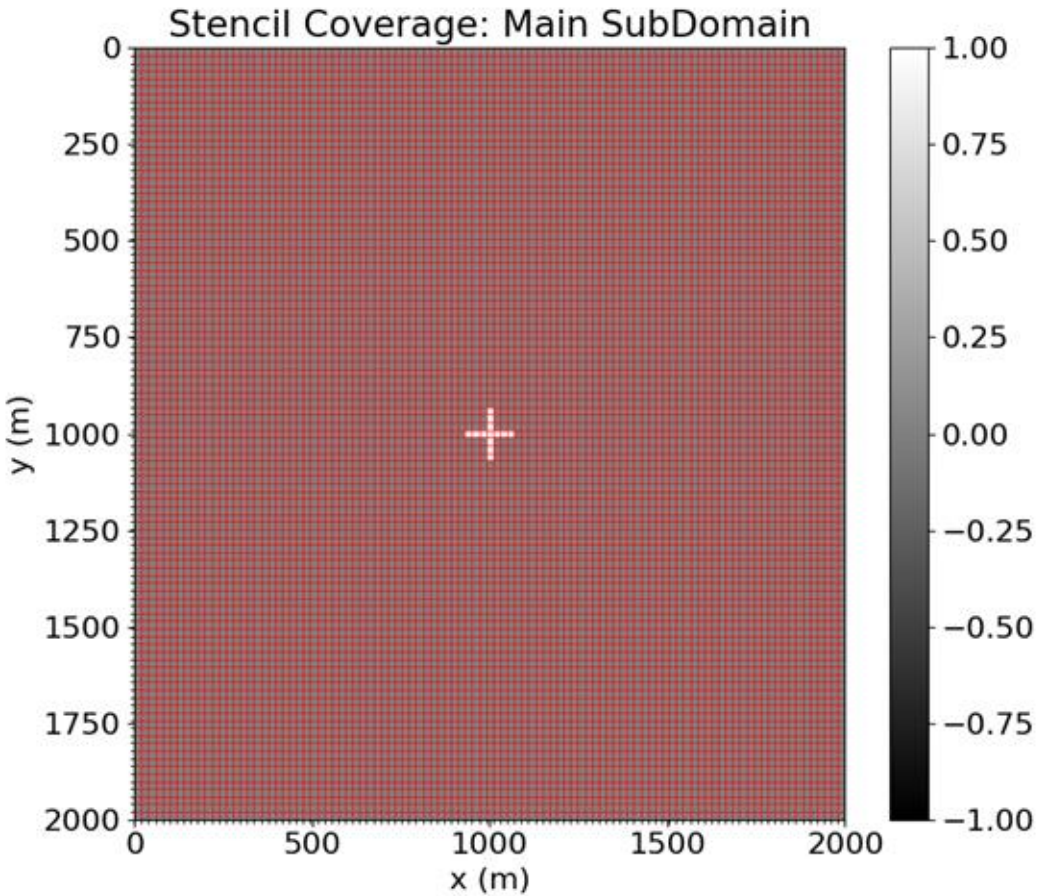
vy Coefficients

	Function	Coefficient	Coefficient/h_y
0	v_y(t + dt, x, y + 3*h_y/2)	0.75/h_y	0.037500
1	v_y(t + dt, x, y + 7*h_y/2)	0.0166666667/h_y	0.000833
2	v_y(t + dt, x, y - 3*h_y/2)	0.15/h_y	0.007500
3	v_y(t + dt, x, y - h_y/2)	-0.75/h_y	-0.037500
4	v_y(t + dt, x, y - 5*h_y/2)	-0.0166666667/h_y	-0.000833
5	v_y(t + dt, x, y + 5*h_y/2)	-0.15/h_y	-0.007500

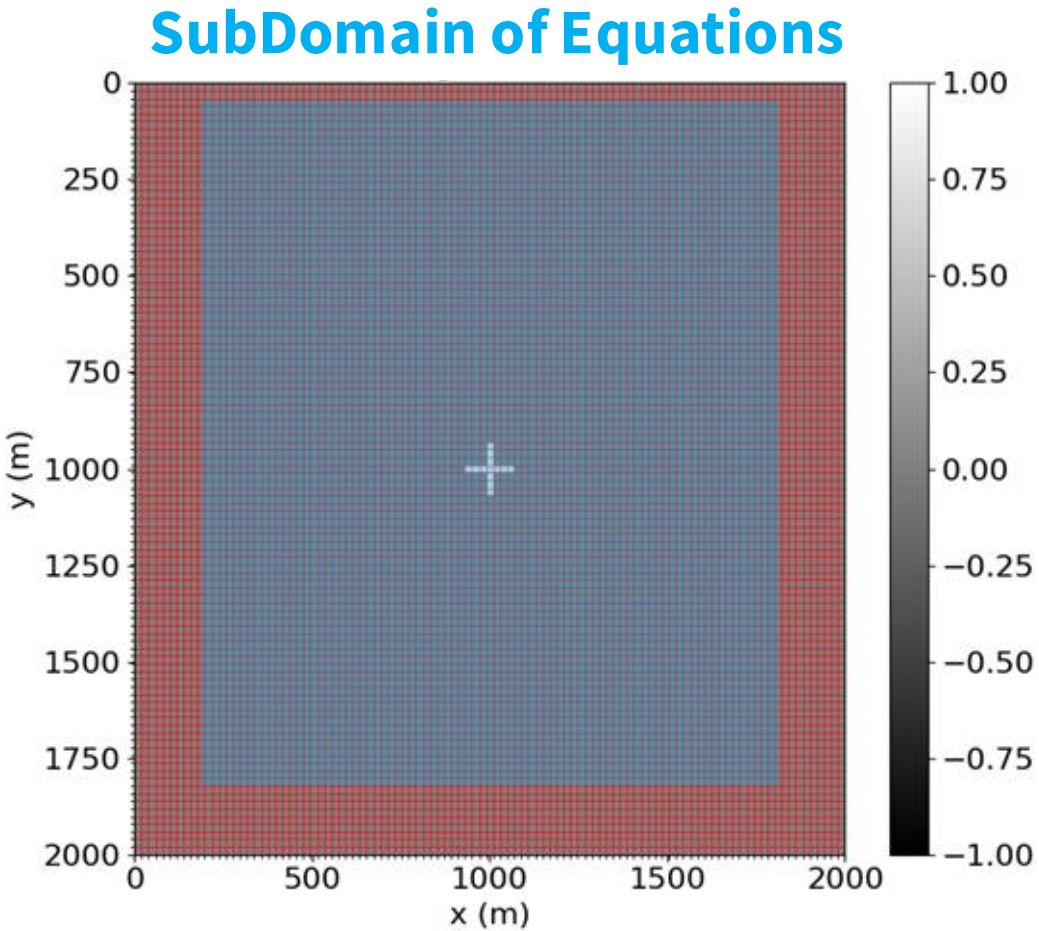
Examining Stencils

```
1  from devito import Eq, grad, div
2
3  eq_v = Eq(v.forward, v - dt*grad(p)/density, subdomain=main)
4
5  eq_p = Eq(p.forward, p - dt*velocity**2*density*div(v.forward), subdomain=main)
```

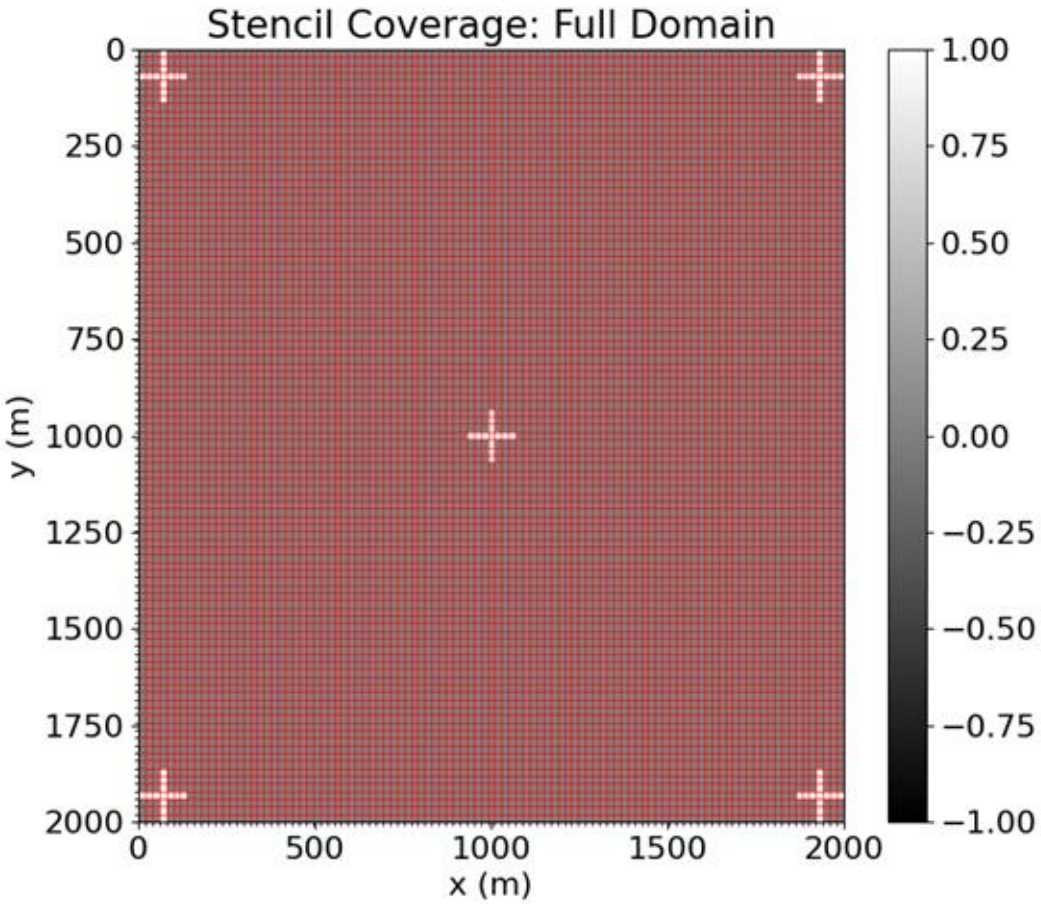
Acoustic: Impulse Responses



Acoustic: Impulse Responses

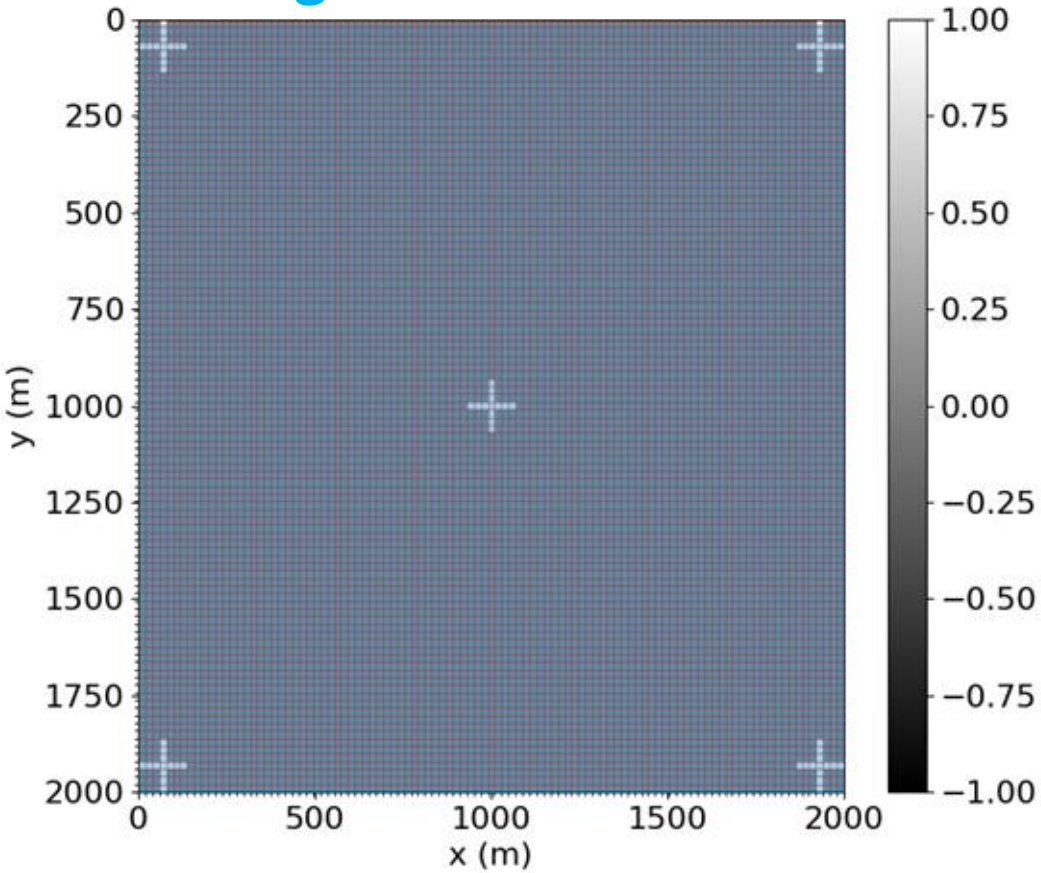


Acoustic: Impulse Responses



Acoustic: Impulse Responses

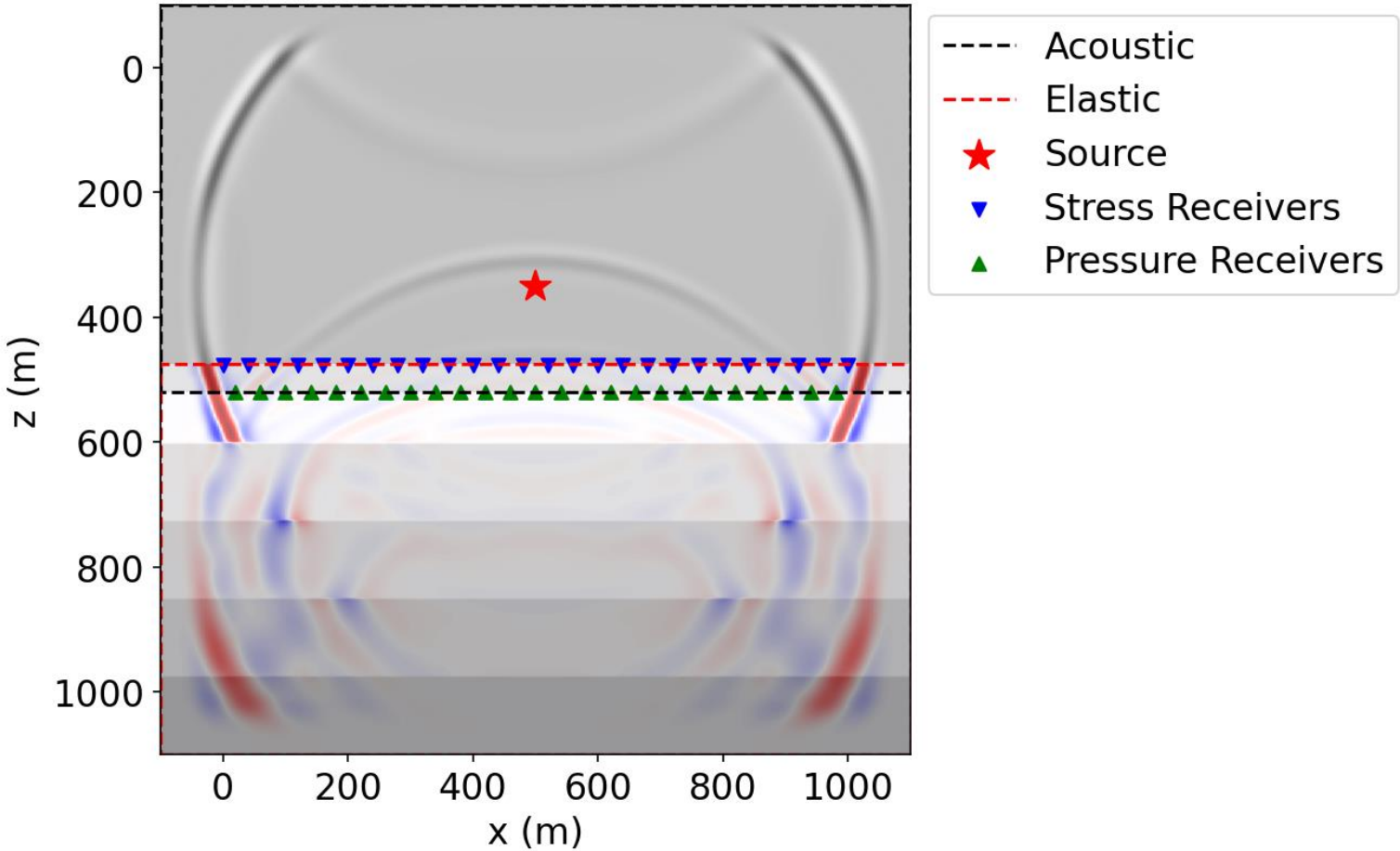
Changed the SubDomain



Validation and quality control

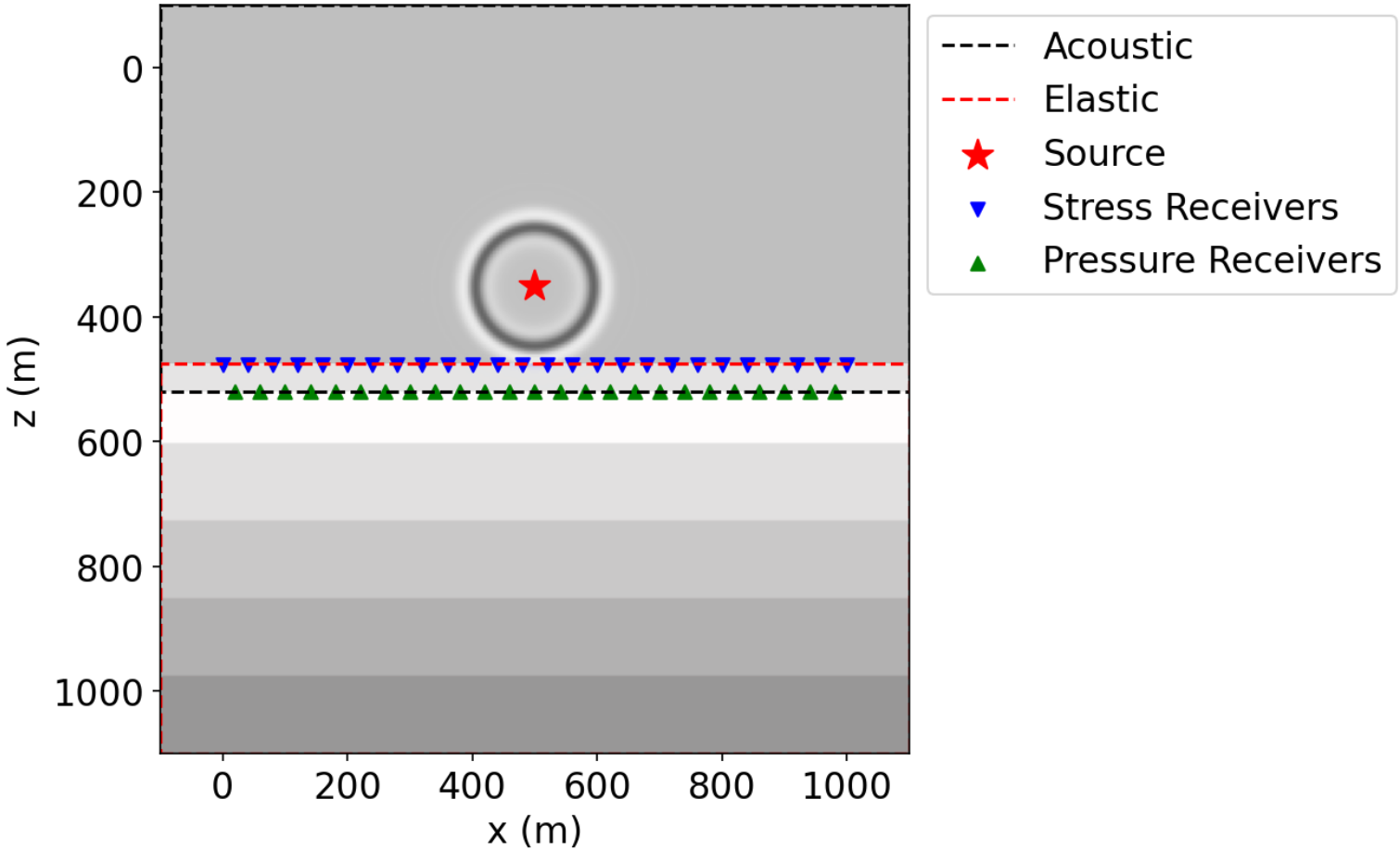
- Stability
  - Positioning (indexing and domain decomposition)
  - Stencils
  - CFL
- Result analysis
  - Are the results consistent with expectations
    - Boundaries
    - Changes in source and receiver (filter function) locations
    - Receiver data agrees with final wavefields (state parameters)
    - Changes in medium parameters

Sparse Functions: Inject & Interpolate

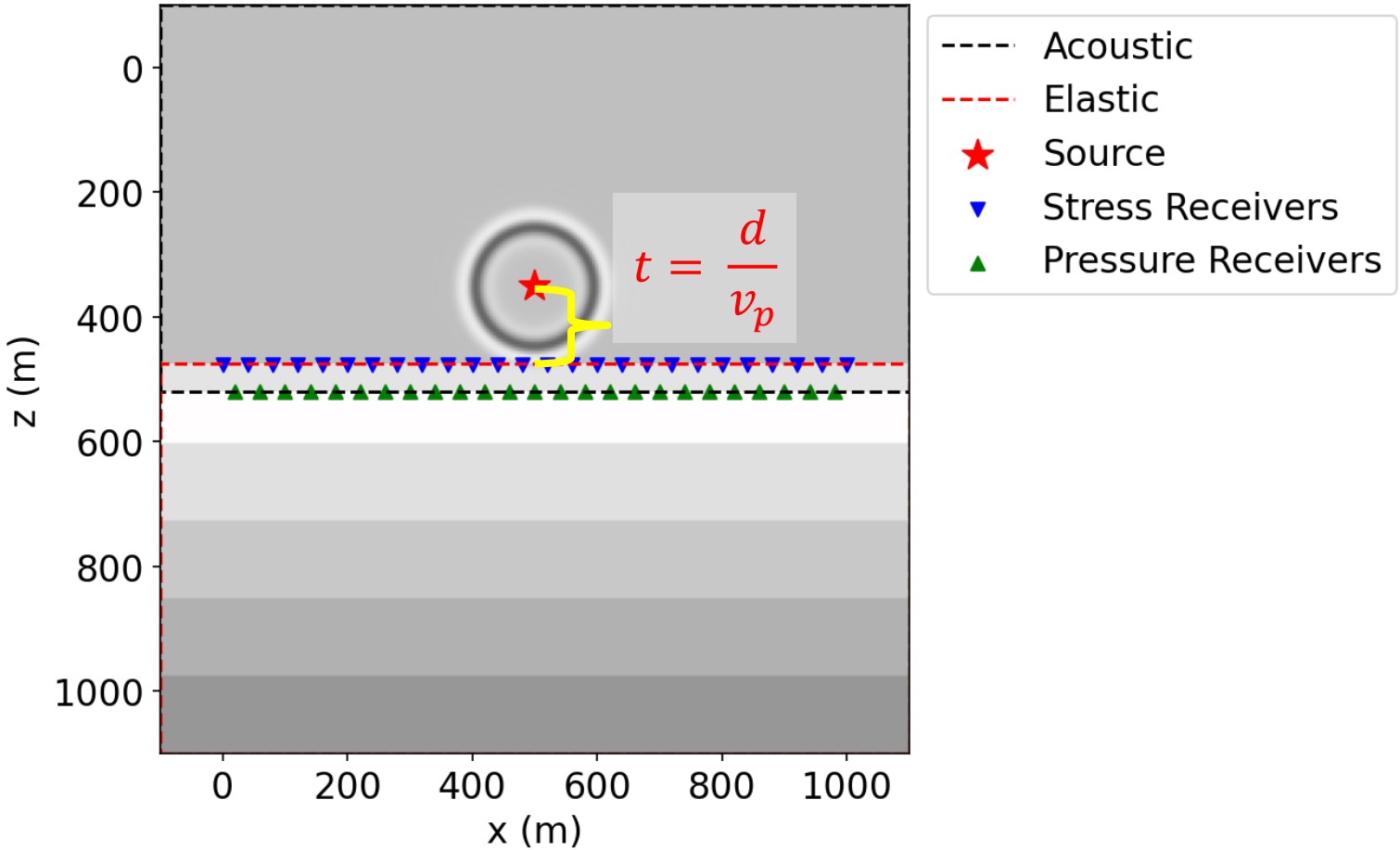


notebooks/Forcing_Filtering_Function_QC_07_functions_on_subdomains.ipynb

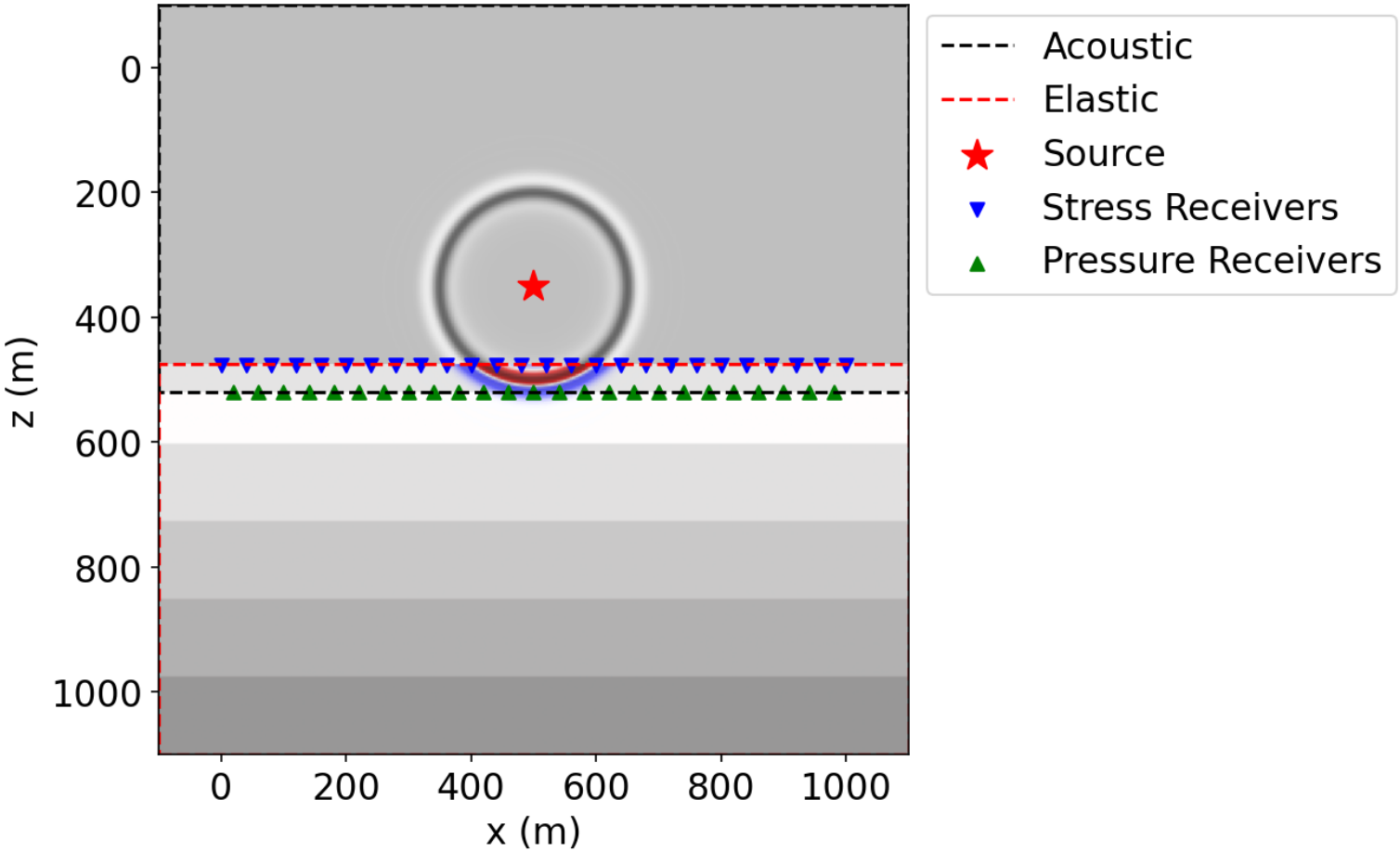
Sparse Functions: Calculate Time to Receivers



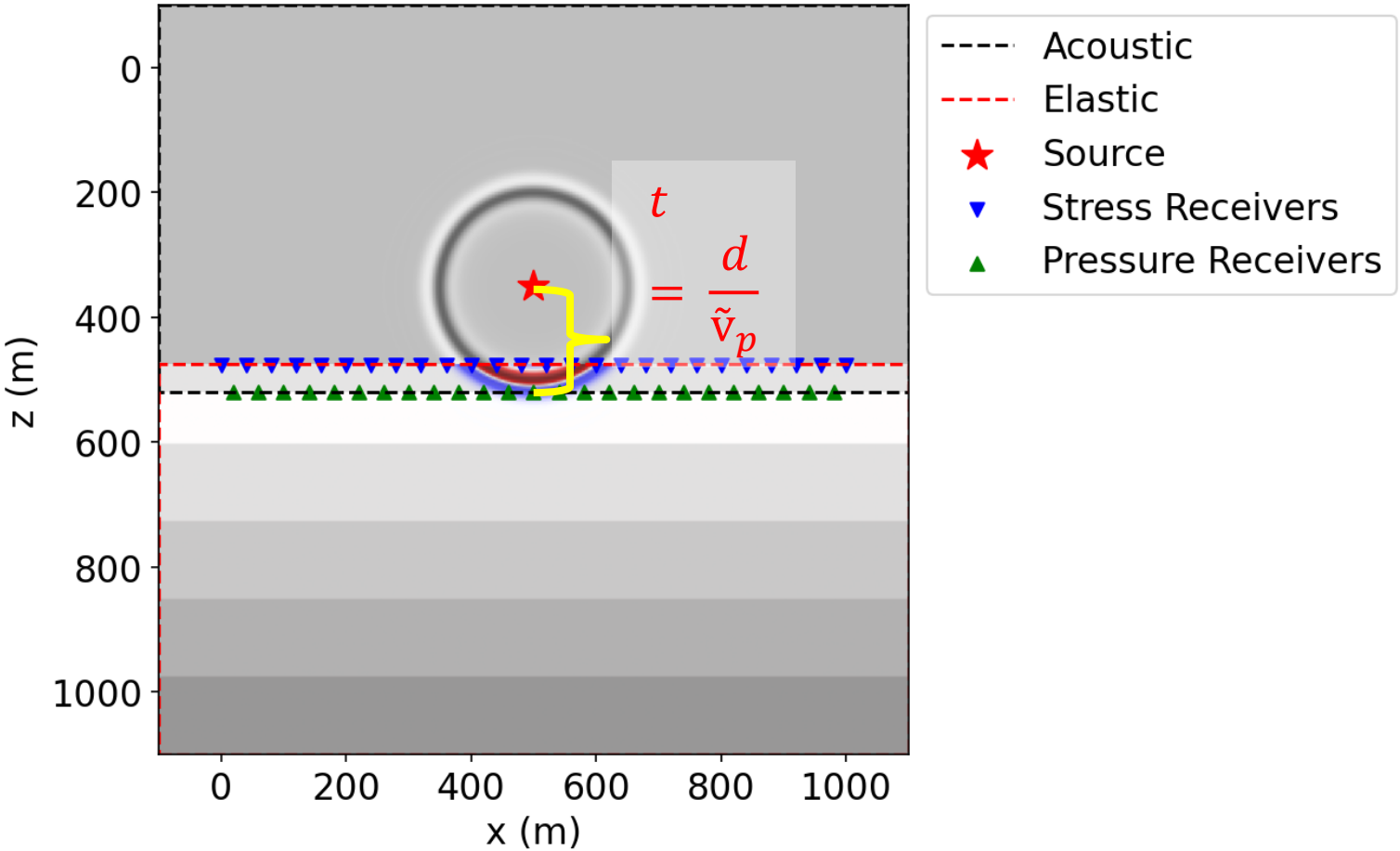
Sparse Functions: Calculate Time to Receivers



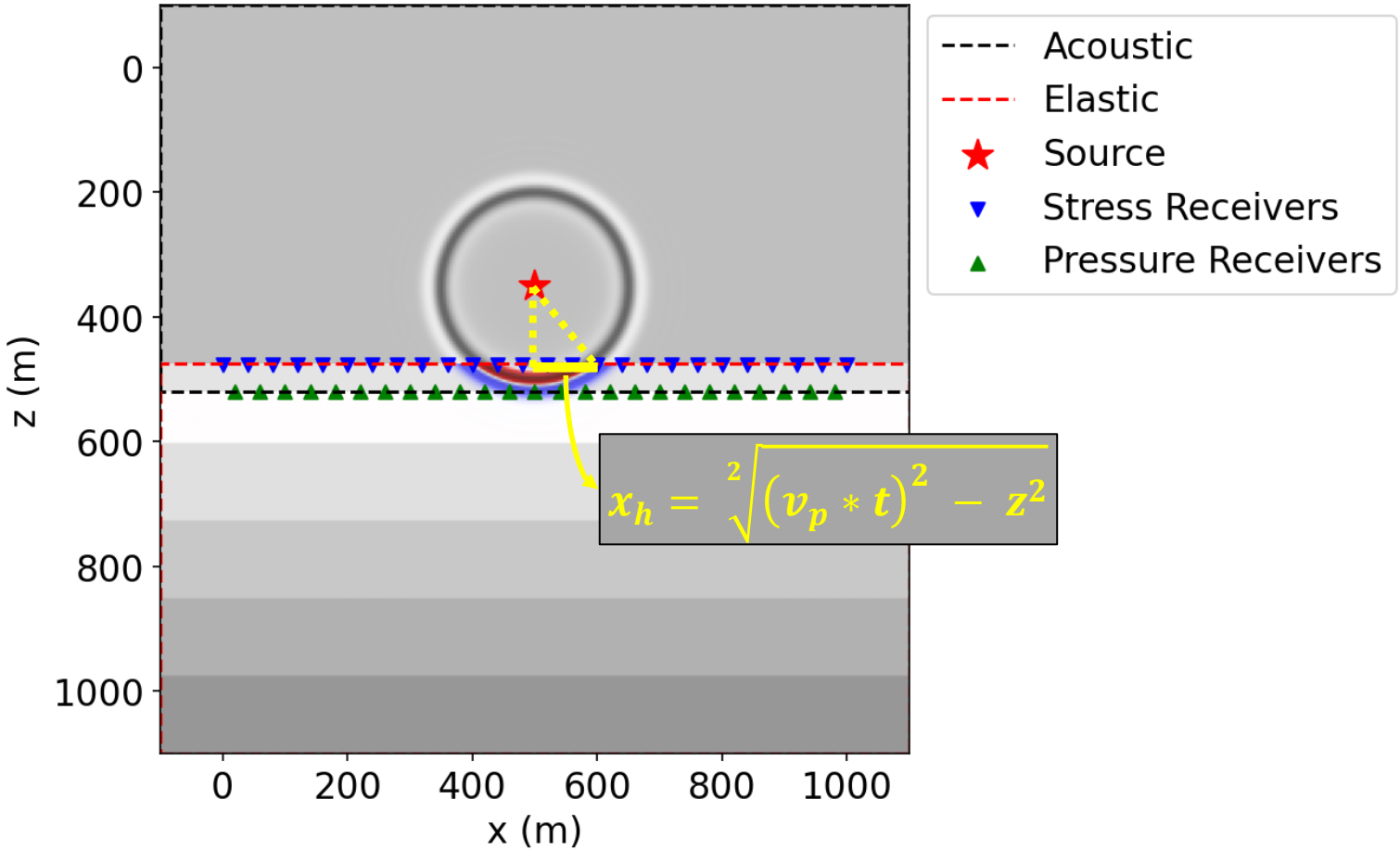
Sparse Functions: Calculate Time to Receivers



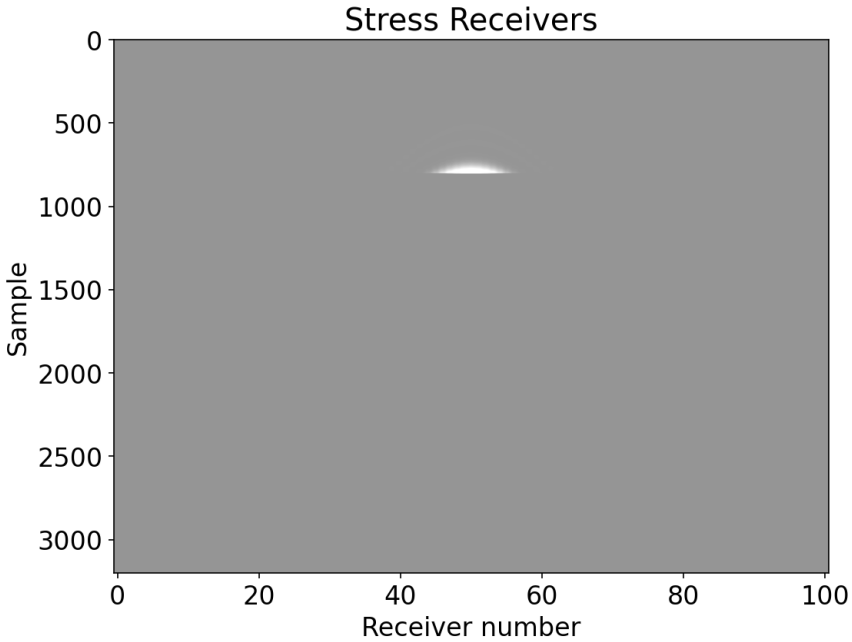
Sparse Functions: Calculate Time to Receivers

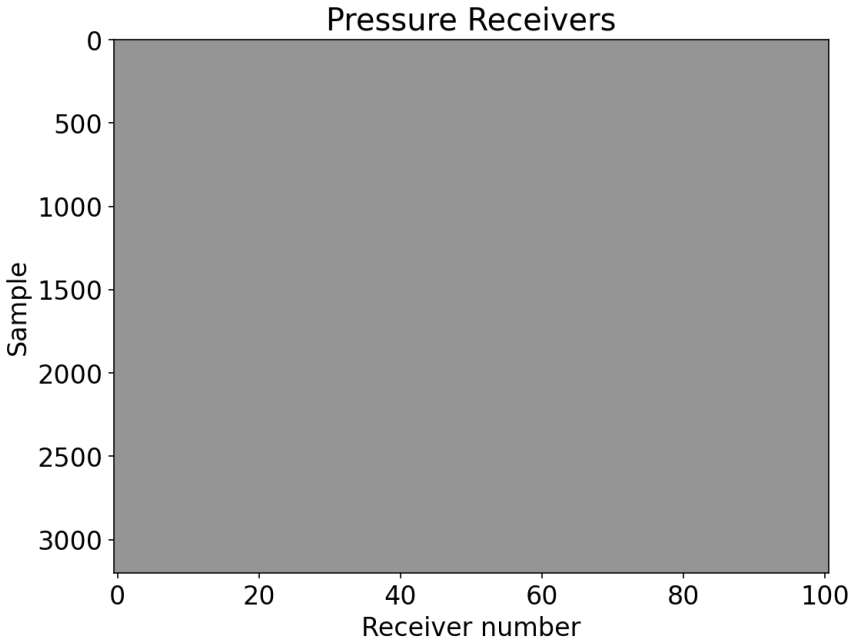


Sparse Functions: Calculate Time to Receivers

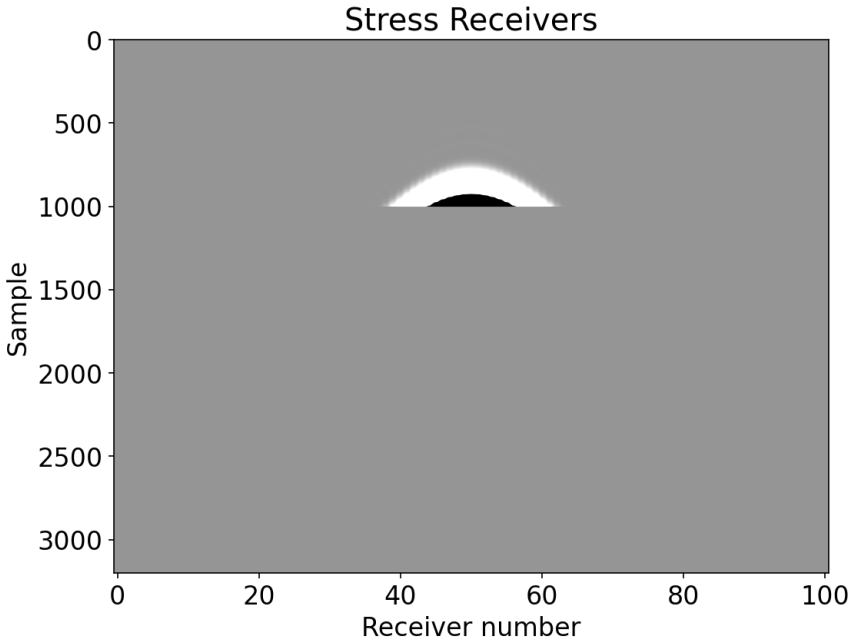


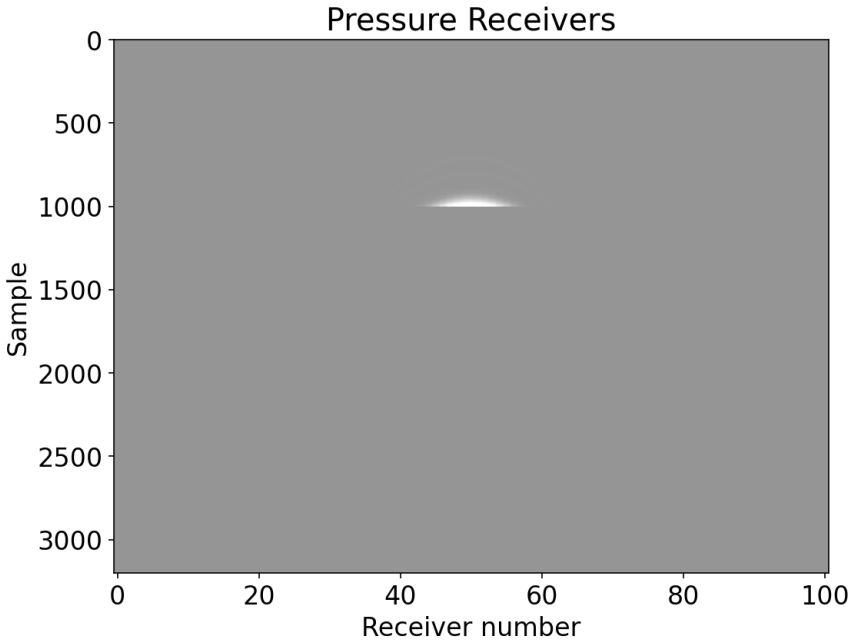
Sparse Functions: First Arrival (*Top of Elastic SubDomain*)



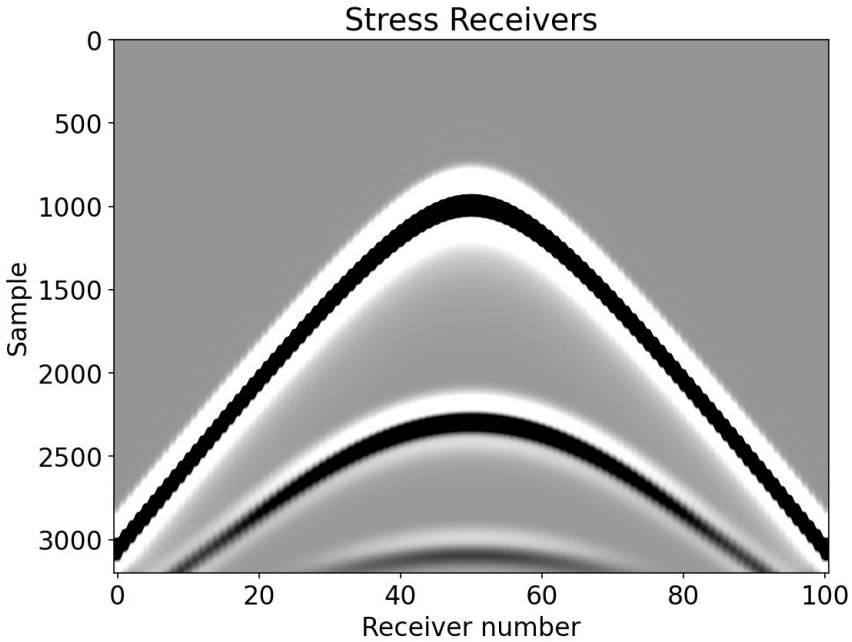


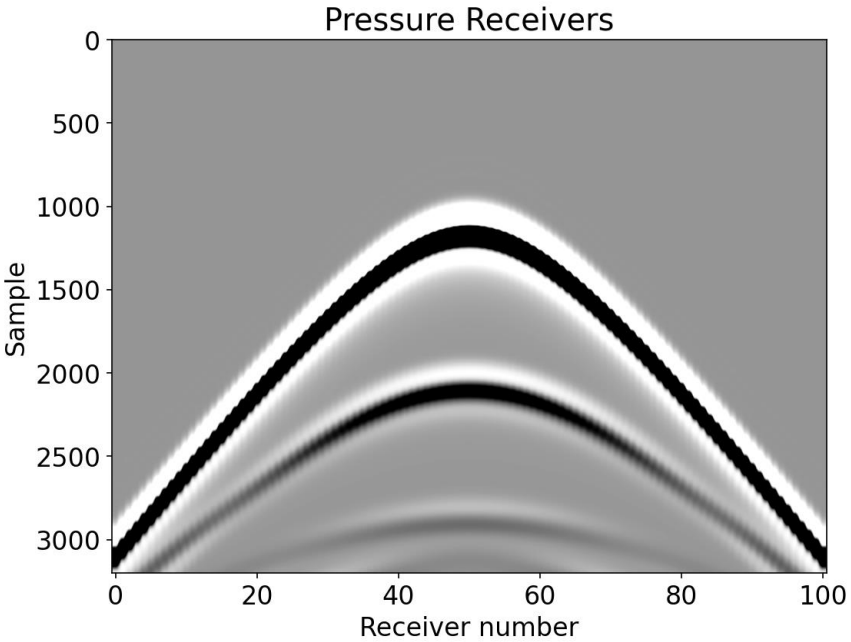
Sparse Functions: First Arrival (*Bottom of Acoustic SubD.*)



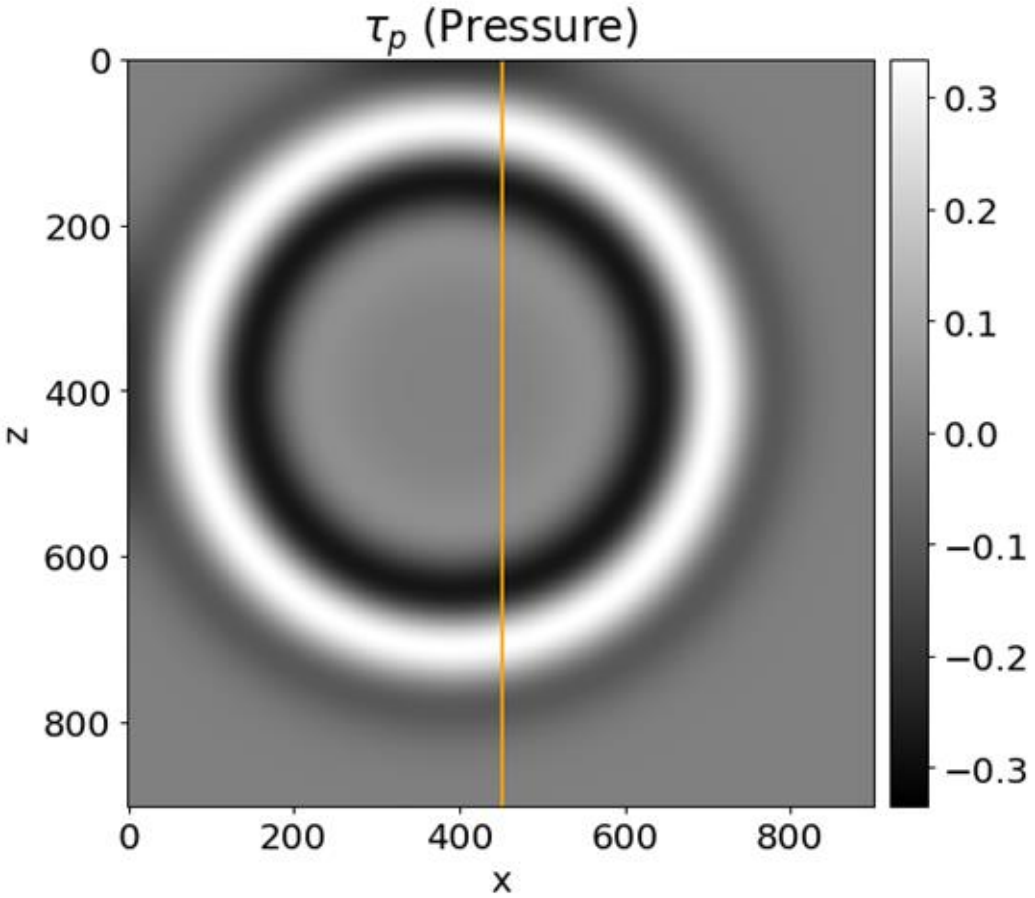


Sparse Functions: Full Data (all offsets)

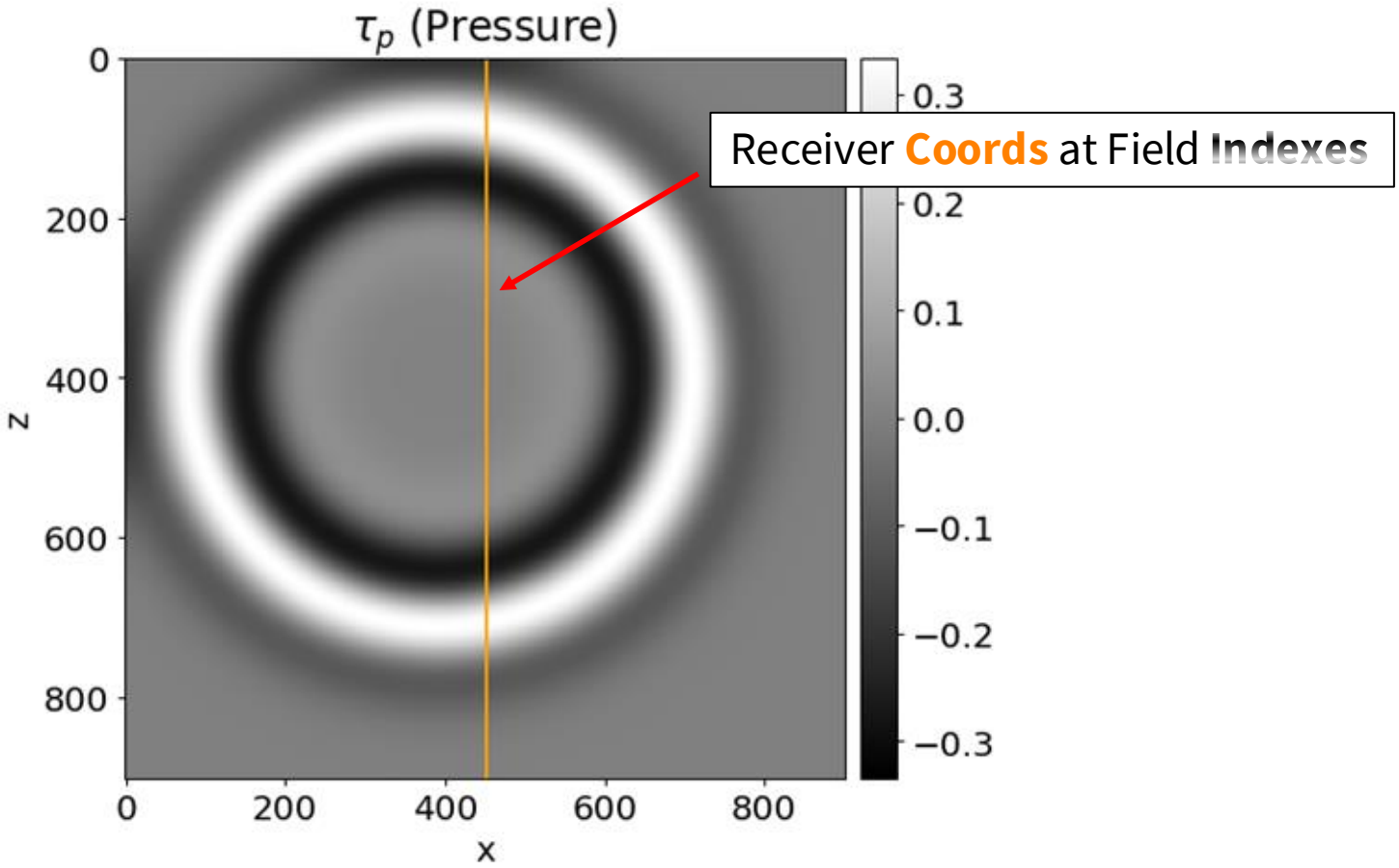


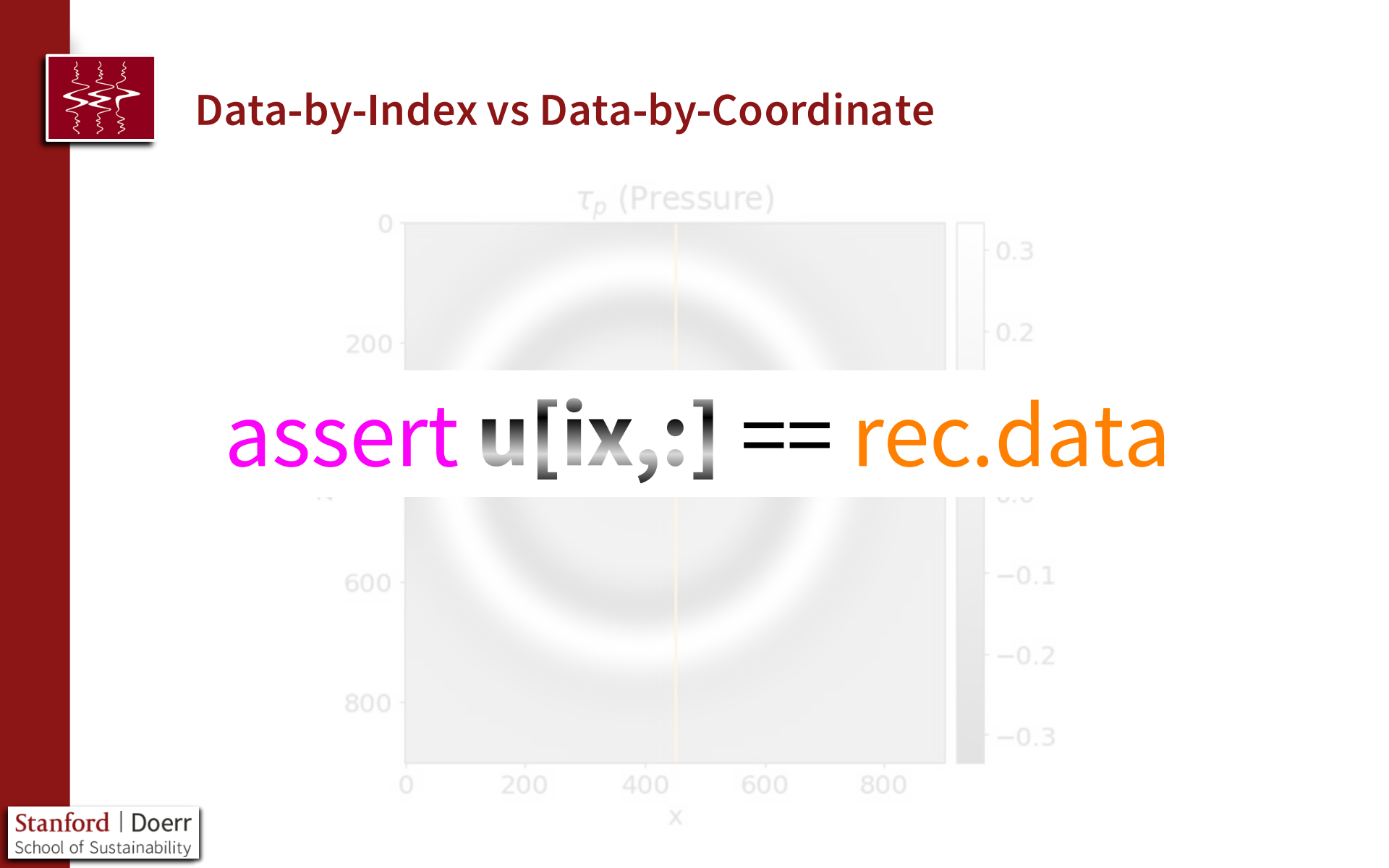


Function-Field Vs Sparse-Function Interpolate



Function-Field Vs Sparse-Function Interpolate

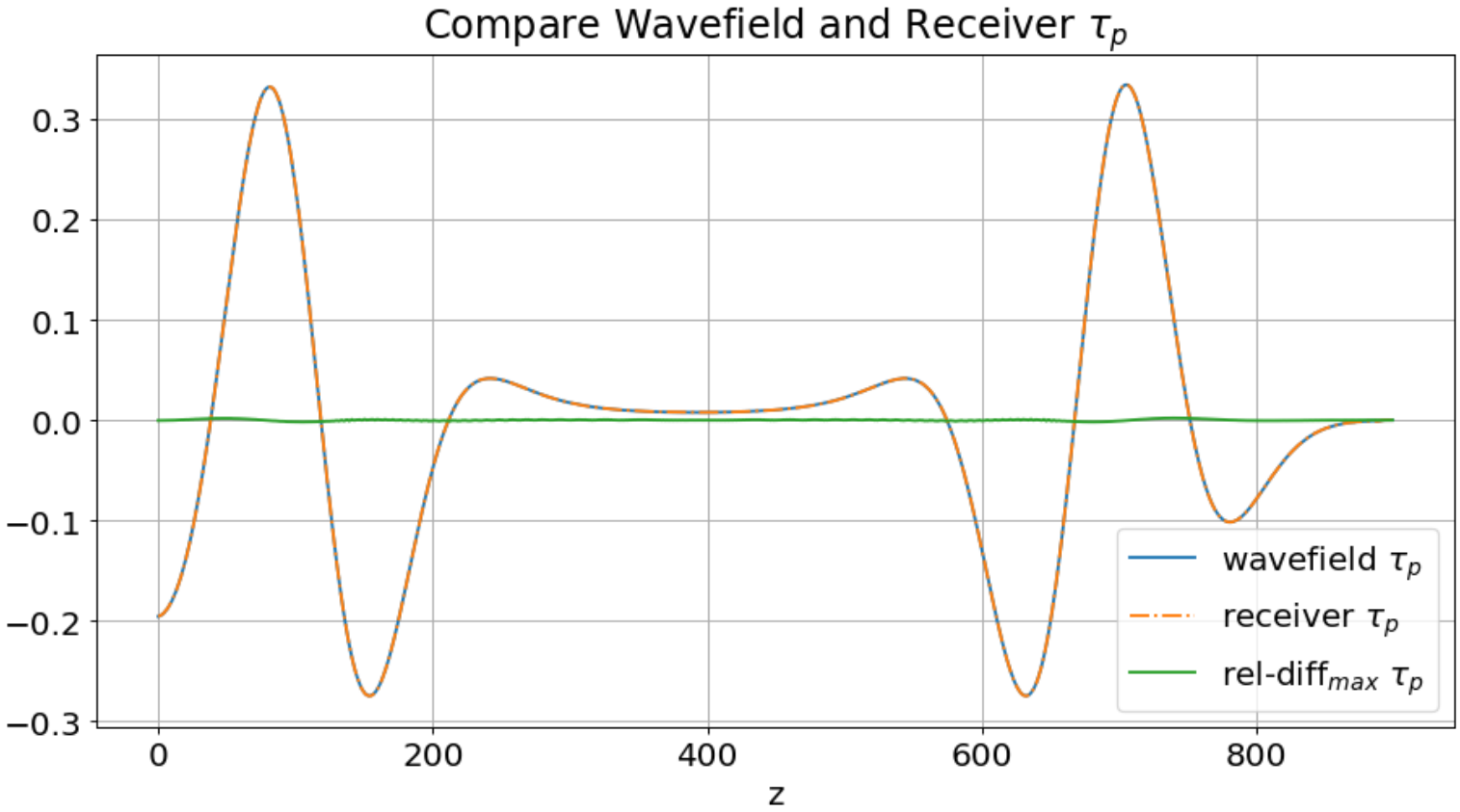




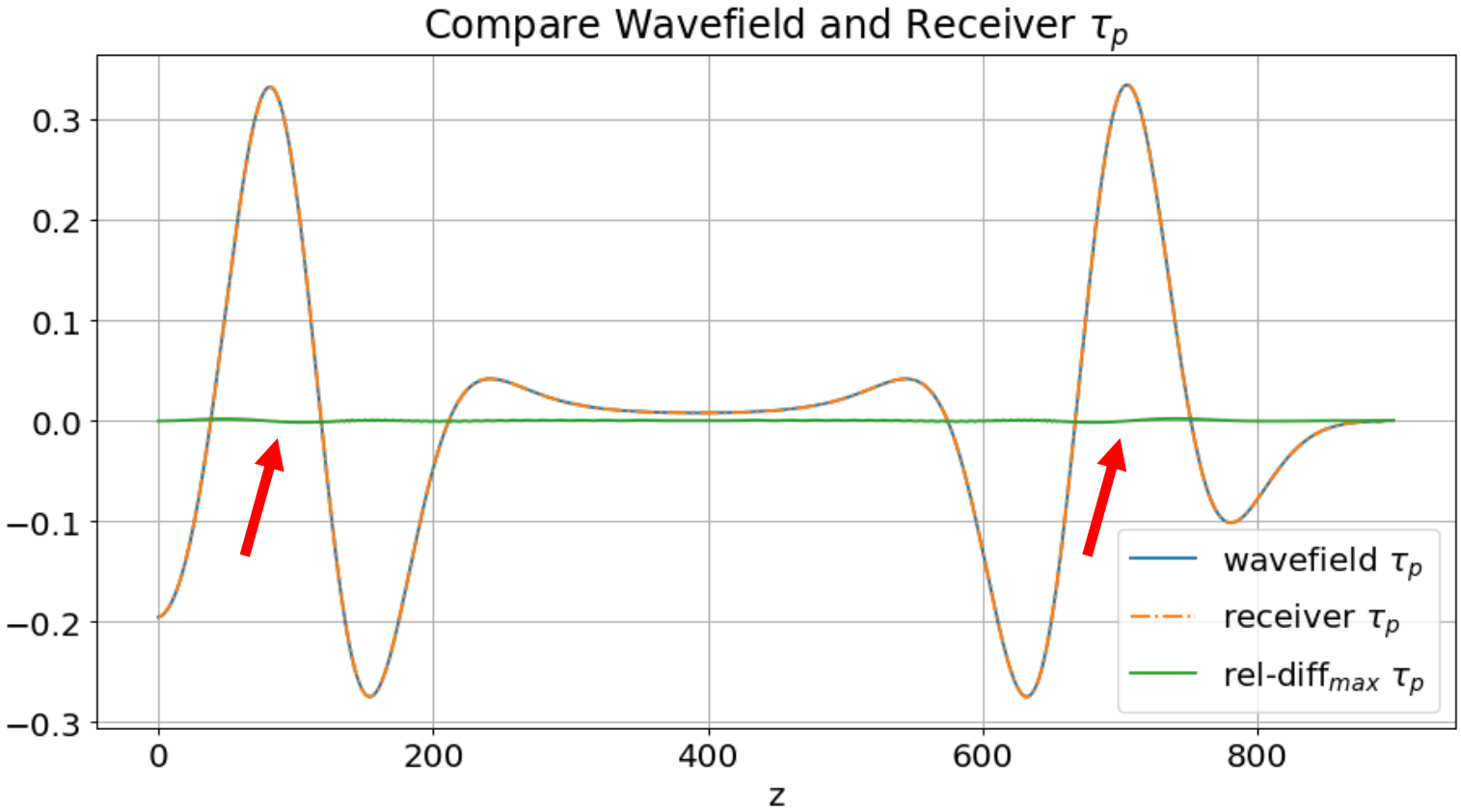
Data-by-Index vs Data-by-Coordinate


```
assert u[ix,:] == rec.data
```

Compare Pressure: Trace



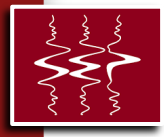
Compare Pressure: Trace



Adapting to Different Development Paradigm

- Validation and quality control
- Integration into HPC workflows
- System-level performance and local optimizations

Acoustic and Elastic Wave Equations



Integration into HPC workflows

- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control

Integration into HPC workflows

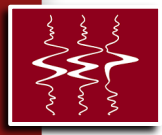
- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control (e.g., loops over operators)

Integration into HPC workflows

- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control (e.g., MPI: calling operators)
  - Cluster (SLURM)
  - Dask
  - Orchestrator (e.g., Airflow): maybe someday…

Integration into HPC workflows

- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control



Integration into HPC workflows

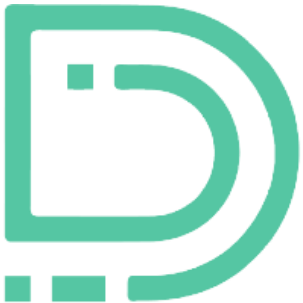
- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control

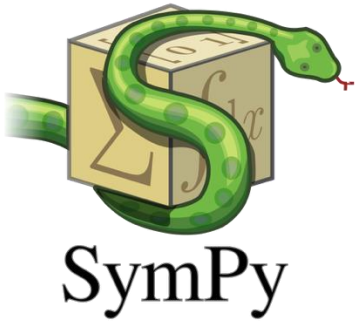
Physics and Numerics

- Acoustic vs elastic (ISO, VTI, TTI)
- Sources and receivers*
- Stencils*
- BCs*

Physics and Numerics

- Acoustic vs elastic (ISO, VTI, TTI)
- Sources and receivers*
- Stencils*
- BCs*





2D Acoustic and Elastic Overlap

```
# Acoustic update
pde_p = p.dt2 - cp**2*p.laplace + damp*p.dt
eq_p = Eq(p.forward, solve(pde_p, p.forward), subdomain=upper)

# Elastic update
pde_v = v.dt - b*div(tau) + damp*v.forward
pde_tau = tau.dt \
    - lam*diag(div(v.forward)) \
    - mu*(grad(v.forward) + grad(v.forward).transpose(inner=False)) \
    + damp*tau.forward

eq_v = Eq(v.forward, solve(pde_v, v.forward), subdomain=lowerfield)
eq_t = Eq(tau.forward, solve(pde_tau, tau.forward), subdomain=lower)

# Coupling: p -> txx, tyy
eq_txx_tr = Eq(tau[0, 0].forward, p.forward, subdomain=uppertransition)
eq_tyy_tr = Eq(tau[1, 1].forward, p.forward, subdomain=uppertransition)
# Coupling: txx, tyy -> p
eq_p_tr = Eq(p.forward, (tau[0, 0].forward + tau[1, 1].forward)/2, subdomain=lowertransition)
```

```
src_term = src.inject(field=p.forward, expr=src)
rec_term = rec.interpolate(expr=tau[0, 0].forward+tau[1, 1].forward)
```

```
op1 = Operator([eq_v, eq_p, eq_t, eq_p_tr, eq_txx_tr, eq_tyy_tr] + src_term + rec_term)
```

Order of terms matters!

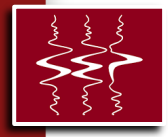
2D Acoustic and Elastic Overlap

```
# Acoustic update
pde_p = p.dt2 - cp**2*p.laplace + damp*p.dt
eq_p = Eq(p.forward, solve(pde_p, p.forward), subdomain=upper)

# Elastic update
pde_v = v.dt - b*div(tau) + damp*v.forward
pde_tau = tau.dt \
    - lam*diag(div(v.forward)) \
    - mu*(grad(v.forward) + grad(v.forward).transpose(inner=False)) \
    + damp*tau.forward

eq_v = Eq(v.forward, solve(pde_v, v.forward), subdomain=lowerfield)
eq_t = Eq(tau.forward, solve(pde_tau, tau.forward), subdomain=lower)

# Coupling: p -> txx, tyy
eq_txx_tr = Eq(tau[0, 0].forward, p.forward, subdomain=uppertransition)
eq_tyy_tr = Eq(tau[1, 1].forward, p.forward, subdomain=uppertransition)
# Coupling: txx, tyy -> p
eq_p_tr = Eq(p.forward, (tau[0, 0].forward + tau[1, 1].forward)/2, subdomain=lowertransition)
```

```
src_term = src.inject(field=p.forward, expr=src)
rec_term = rec.interpolate(expr=tau[0, 0].forward+tau[1, 1].forward)
```

```
op1 = Operator([eq_v, eq_p, eq_t, eq_p_tr, eq_txx_tr, eq_tyy_tr] + src_term + rec_term)
```

Order of terms matters!



3D Elastic Example with Free-surface

```
src = RickerSource(name='src', grid=grid, f0=f0, time_range=time_range, interpolation='sinc')
src.coordinates.data[:] = 0.5*ex
src.coordinates.data[:, -1] = 0.
```

```
# The source injection term
src_xx = src.inject(field=tau.forward[0, 0], expr=src)
src_yy = src.inject(field=tau.forward[1, 1], expr=src)
src_zz = src.inject(field=tau.forward[2, 2], expr=src)
```

← Isotropic Moment Tensor

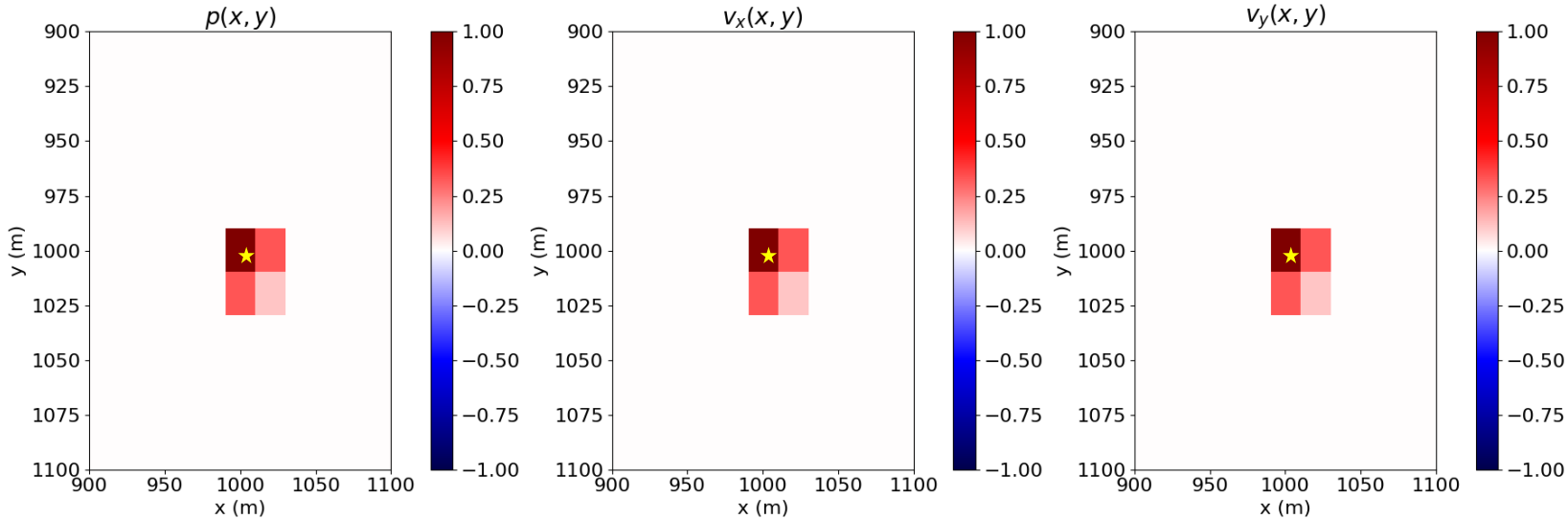
```
# Note: v free surface equations inserted before tau update
op = dv.Operator([u_v] + eqs_fs_v + [u_tau] + src_xx + src_yy + src_zz + eqs_fs_tau)
```

rec

Source (Forcing) Function: Injection Off-Grid

h = 20 m

```
ptsrc.coordinates.data[0, :] = 1005.
```

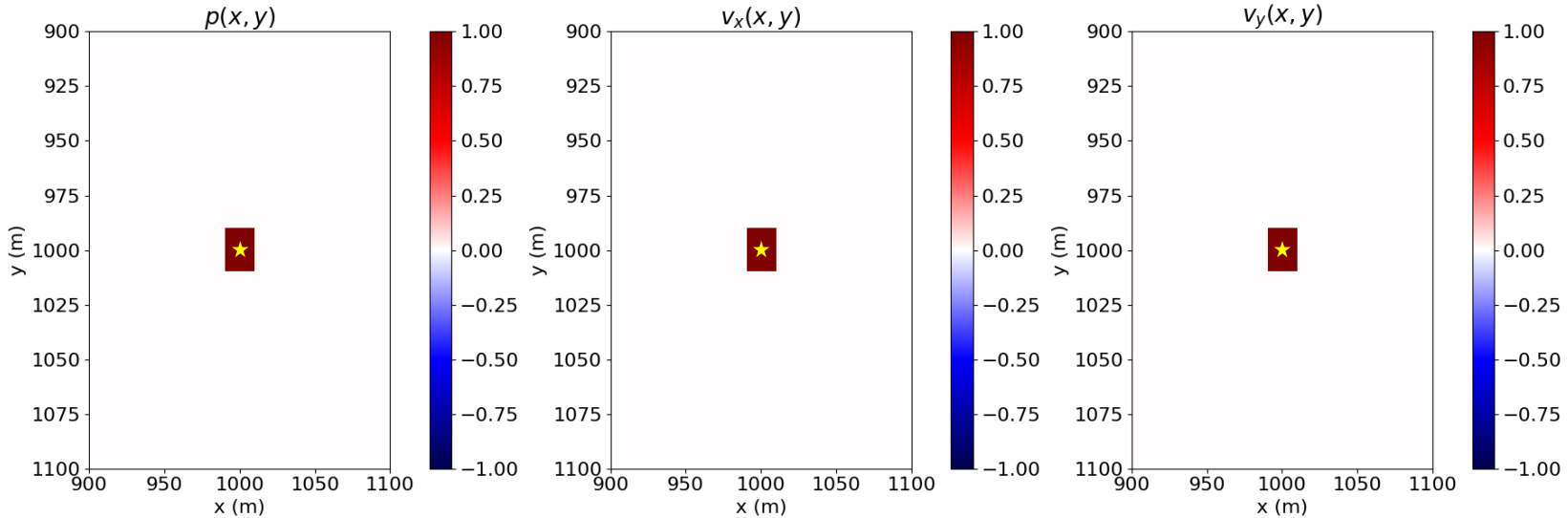


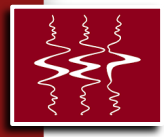
notebooks/Forcing_Function_QC_Curiosity_04_boundary_conditions.ipynb

Source (Forcing) Function: Injection Off-Grid

h = 20 m

```
ptsrc.coordinates.data[0, :] = 1000.
```



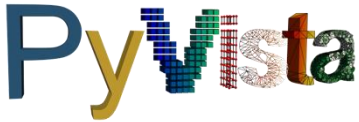


Integration into HPC workflows

- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control

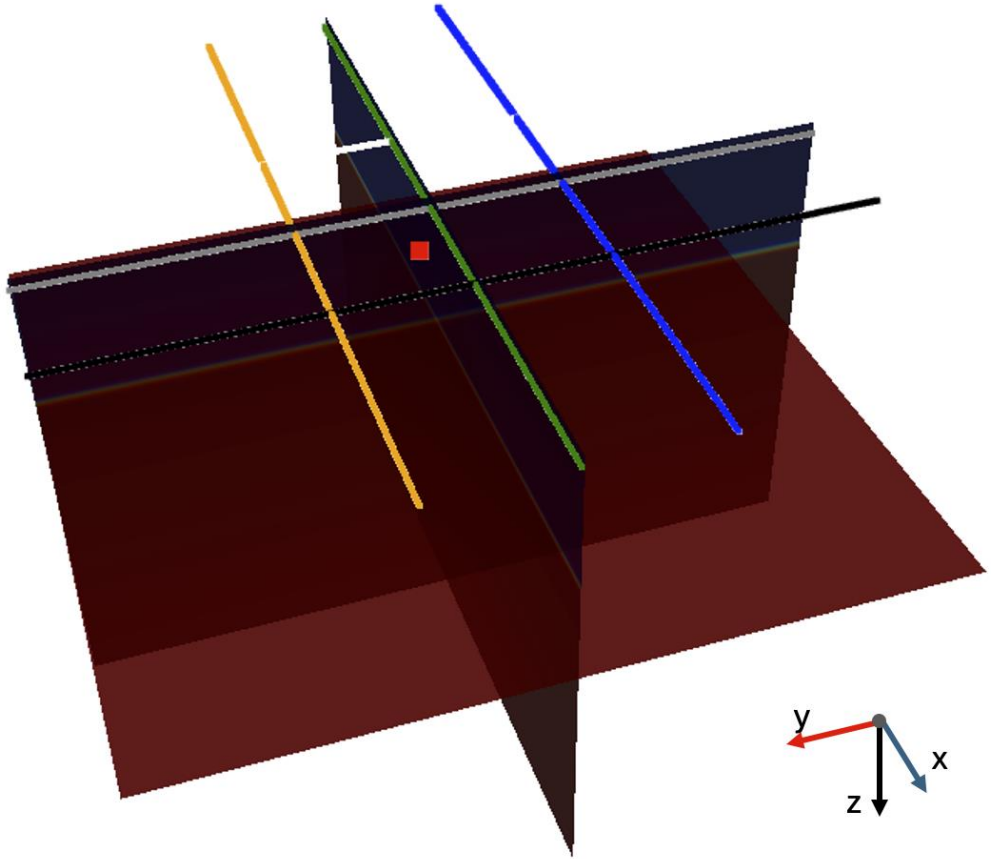
Data Processing and Visualization

- Python: processing and visualization
- Jupyter Notebooks: development, testing, and visualization



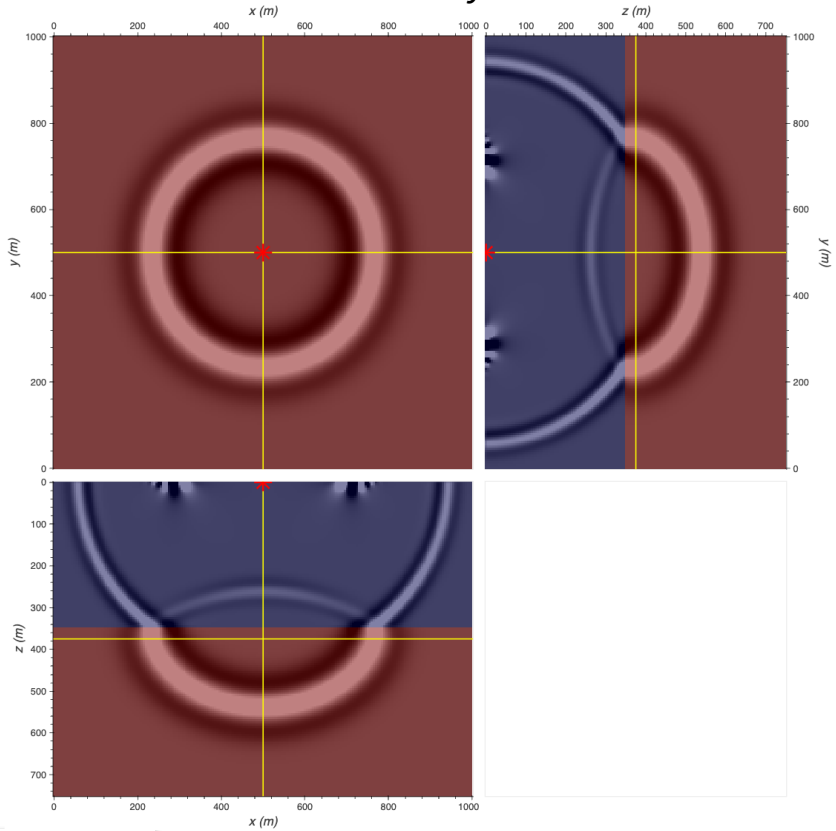
Visualization Tools (Python + Notebooks)

PyVista

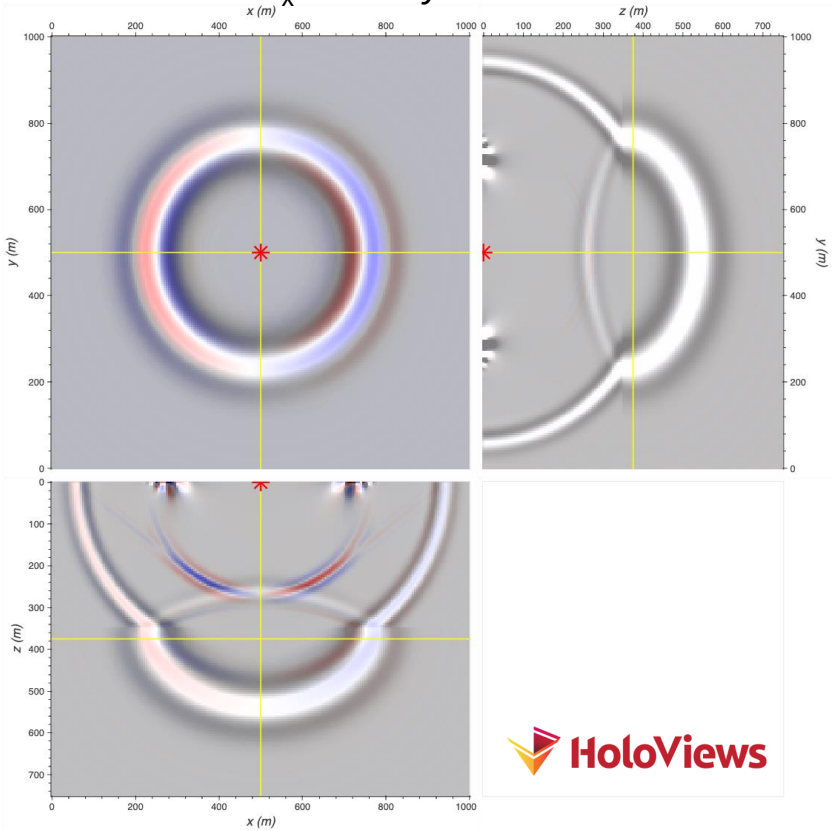


Visualization Tools (Python + Notebooks)

Pressure Overlays Model



v_x Overlays Pressure



Integration into HPC workflows

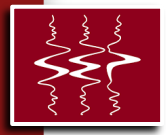
- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control

Integration into HPC workflows

- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control

Integration into HPC workflows

- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control



Integration into HPC workflows

- Physics and numerics
- Data processing and visualization
- Package encapsulation (modules, classes, recipes, etc.)
- High-level algorithm performance and control

Package Encapsulation Goals/Constraints

- Usability
- Extendibility
- Reproducibility
- Testability

Package encapsulation

Thank You

References

- [Bisbas, G., et al., *Automated MPI-X code generation for scalable finite-difference solvers*, in *Proc. IEEE IPDPS*, 2025
- [Micikevicius, P., *3D finite difference computation on GPUs using CUDA*, GPGPU-2 2009]

repo